



University
of Stavanger

FILIP B. GOTTEN, RASMUS ØGLÆND, AND TORJUS J. KNUDSEN

DEPARTMENT OF ELECTRICAL ENGINEERING AND COMPUTER SCIENCE

Enhancing Bandwidth Incentive Simulation for Swarm: Recreating a Python-based Tool in Go

Bachelor's Thesis - Computer Science - May 2023

I, **Filip B. Gotten, Rasmus Øglænd, and Torjus J. Knudsen** ,
declare that this thesis titled, “Enhancing Bandwidth Incentive Simulation for
Swarm: Recreating a Python-based Tool in Go” and the work presented in it are
my own. I confirm that:

- This work was done wholly or mainly while in candidature for a master’s
degree at the University of Stavanger.
- Where I have consulted the published work of others, this is always clearly
attributed.
- Where I have quoted from the work of others, the source is always given.
With the exception of such quotations, this thesis is entirely my own work.
- I have acknowledged all main sources of help.

Abstract

This thesis builds upon an existing Python simulation tool that models the behavior of bandwidth incentives in Swarm, forming the basis for our research.

Recognizing the potential for improvements, we undertook the challenge of recreating the existing tool in Go, aiming to enhance its performance and overall functionality. This thesis details the original design of the Python-based tool, the subsequent advancements made in our adjusted design, and specific implementation differences involved when developing the Go-based tool.

The primary focus of our work was to utilize the inherent advantages of Go, particularly its support for concurrency, to achieve enhanced performance in simulating bandwidth incentives. We also addressed various shortcomings of the original tool, incorporating implementation enhancements to better simulate Swarm and improve upon the overall quality and usability of the simulation tool.

In the evaluation phase of our study, we compare the execution times of the original Python-based simulation tool with the newly developed Go-based tool. Our findings reveal a significant improvement in performance, with the Go implementation outperforming the Python version by a factor of up to 10,000.

This paper contributes to the field of decentralized storage systems by highlighting the benefits of employing Go for simulation tasks, especially in scenarios that demand high performance. Our work provides valuable insights into the design considerations, implementation choices, and performance gains associated with recreating a simulation tool in Go for bandwidth incentives in Swarm.

Acknowledgements

We as a team would like to thank our supervisor, Leander Jehl, for his outstanding supervision and advice during this project. He has helped us overcome several obstacles along the way and influenced the direction of our research with his vast expertise and understanding of the field. His helpful criticism and insightful comments have greatly improved the quality of our work.

We would also like to extend our sincere thanks to Vahid Heidari pour, who was responsible for implementing and developing the original Python simulation tool that served as the foundation for our paper. His help has enabled us to conduct our research effectively and efficiently. We are grateful for his dedication and commitment to this project, which has made a significant contribution to our success.

Last but not least, we want to thank each other as a team. We could not have done what we did without each other's support and effort, and working together has been an exceptionally educational and fulfilling experience. We are appreciative of the chance to collaborate with one another and are satisfied with what we have accomplished.

Contents

Abstract	iii
Acknowledgements	v
1 Introduction	1
1.1 Objectives	3
1.2 Approach	3
1.3 Contributions	4
1.4 Outline	4
2 Background	5
2.1 The Ethereum Swarm Network	6
2.1.1 Swarm data sharing steps	6
2.1.2 Routing in Swarm	7
2.1.3 Incentives driving swarm	9
2.2 Go vs Python	12
2.3 Concurrency in Go	13
2.3.1 Go Concurrency Primitives	13
2.3.2 Producer-Consumer Pattern	14
3 Original Design	17
3.1 Design With CadCAD	17
3.2 PySwarmSim Simulation Steps	18
3.2.1 Initialize the Network	19
3.2.2 Request Creation	20
3.2.3 Find Routes	22
3.2.4 Update	23

3.2.5	Extract and Analyze Data	27
3.3	Conclusion on Original Design	27
4	Adjusted Design	29
4.1	Design Without cadCAD	29
4.1.1	Replicating the needed functionality from cadCAD	30
4.1.2	New design without cadCAD's limitations	30
4.2	Adjusting the design to implement concurrency	31
4.2.1	Concurrency inside routing	32
4.2.2	Concurrent request and routing - Naive approach	33
4.2.3	Concurrent request and routing - Global lock approach	34
4.2.4	Making routing and update logic thread-safe	36
4.3	Introducing Goroutine Workers	38
4.3.1	RequestWorker	39
4.3.2	RoutingWorkers	41
4.4	Introducing Epochs	42
4.5	Extract Data	43
4.5.1	StateFlushWorker	43
4.5.2	RouteFlushWorker	44
4.6	Analyzing Data	45
4.6.1	OutputWorker	45
4.7	Final GoSwarmSim Design	46
4.7.1	GoSwarmSim Simulation Steps	46
4.7.2	GoSwarmSim Design	47
4.8	Conclusion on Adjusted design	47
5	Implementation	49
5.1	Challenges when Translating Python Logic	49
5.2	Referencing Objects	50
5.2.1	Value objects vs pointers in Go	51
5.2.2	Drawbacks of pointers	51
5.3	Improving code clarity and readability	52
5.3.1	Objects of variable lengths containing multiple types	52
5.3.2	Route list to requestResult struct	53
5.4	Finding Next-hop Node	54

5.5	Forgiveness of Edge Debt	55
5.5.1	Forgiving during routing	56
5.5.2	Frequency of checking threshold	57
5.5.3	Epoch in forgiveness	58
5.6	Cache	58
5.6.1	Atomic maps	59
5.6.2	Local objects on nodes	59
5.7	Waiting and Retry	60
5.7.1	Pending queue of chunks	61
5.7.2	Retry with alternative routes	62
5.7.3	Waiting and retry together	65
5.8	Calculation of Responsible Nodes	66
5.8.1	Redundant ad-hoc calculations	66
5.8.2	Precomputation and improving algorithm	67
5.8.3	Interval size returned from binary search	67
5.8.4	Final approach	69
5.9	Experiments Configuration	70
5.10	Conclusion on Implementation Improvements	71
6	Evaluation	73
6.1	Validation	73
6.2	Performance Review	74
6.2.1	PySwarmSim vs GoSwarmSim	74
6.2.2	Different versions of GoSwarmSim	78
6.2.3	Finding the optimal number of goroutines	81
6.2.4	Larger Networks	83
6.3	Conclusion on Evaluation	86
7	Discussion	87
7.1	GoSwarmSim From Scratch	87
7.2	Future Work	88
8	Conclusion	91
A	Instructions to Compile and Run System	93

Chapter 1

Introduction

An increasingly popular alternative to traditional centralized storage technology is decentralized peer-to-peer (p2p) storage networks [28]. Decentralized storage relies on a network of multiple independent peers that collaborate to store and send small chunks of data, contrary to having data stored on centralized servers that are managed by a single entity [16]. The increased security provided by decentralized storage is one of its advantages. Data is spread among multiple peers, which makes gaining unauthorized access much more challenging [4]. Decentralized storage networks also eliminate the issue of a single point of failure and typically also have pricing that is much lower than those of their centralized alternatives [28]. It is therefore a desirable choice for people and companies looking for a cheaper substitute to the more expensive centralized storage alternatives [4].

Even with these advantages, decentralized storage networks do have some drawbacks. Incentivizing peers to participate in the network is one of several challenges. Decentralized storage networks require independent peers to operate and maintain the network, hence it is critical to provide these peers with an incentive to join. Peers may be less likely to participate in the network if the rewards for joining the network are unappealing, which could affect the network's safety and dependability [16]. Another limitation with these networks arises when anyone is allowed to join. In such cases, the network risks being compromised due to problems like churn, spamming, and free riding [18].

Developers have started to use token-based incentives to encourage participation in decentralized storage networks to solve these problems [16]. Token-

based incentives reward peers for carrying out specific network operations, such as storing and retrieving data. These rewards are often given out as tokens that may be exchanged for other digital assets on cryptocurrency exchanges. To better understand the effects and mechanisms of token-based incentives in decentralized storage networks, researchers at the University of Stavanger (UiS) have developed a simulation tool written in Python to experiment on *Fair Rewards for Moving Data in Decentralized Storage Networks* [16]. At present, there exist several storage options that utilize p2p networks. However, the Python simulation tool is inspired by characteristics found in Ethereum Swarm [24].

While the Python simulation has been a useful tool for studying incentives in Swarm, it does have some drawbacks. Python may not be the best choice for simulating large-scale networks or performing computationally intensive tasks. The simulation tool had a limitation in that it could only simulate up to a maximum of 1 million requests. Each request in the simulation corresponds to finding a chunk, and in the context of Swarm, each chunk represents 4KB of data [24]. Consequently, simulating one million requests, meaning one million chunks equals a total amount of 4GB of transferred data. If the total number of peers in a Swarm network increases, it is valid to assume the total number of requests for a given time would increase as well. This means the Python simulation tool is lacking in its ability to simulate a large realistic Swarm network. For instance, a network with 10,000 peers and an average upload speed of 12.47MB/s per second equals 420TB of data transferred over the duration of an hour [1]. To simulate the same amount of shared data in the Python simulation tool, considering a total of 420TB of shared data and the same number of peers, a total of 112,230,000,000 requests would be needed in order to find all the chunks.

Python is renowned for its user-friendly syntax and high readability, but it is also acknowledged for its comparatively slower execution speed and the constraint of having all multithreaded threads run on a single CPU core.[2]. Golang, commonly known as Go, on the other hand, is a statically typed, compiled language that focuses on simplicity, and efficiency and has strong support for concurrency [13]. This study aims to recreate the existing Python simulation tool in Go, with the primary objective of evaluating and comparing the performance of the two languages and their respective implementation techniques.

1.1 Objectives

The goals for our thesis are as follows:

- Improve the simulation tools' performance.
- Be able to handle bigger networks and more extensive simulations.
- Make the simulation tool align more closely to the behavior of Swarm.
- Create a more user-friendly simulation tool that offers ease of use and flexibility for future extensions.

1.2 Approach

To achieve our goals, we pursued the following approach:

- Understand the existing simulation framework based on background from research papers.
- Familiarize ourselves with the Kademlia overlay network.
- Reimplement the existing simulation tool using the Go programming language.
- Utilize concurrent programming techniques to improve the simulation's execution time and increase its scalability potential.
- Validate the redesigned simulation tool by comparing it to the original version.
- Evaluate the performance of the redesigned simulation tool by measuring its time, as well as its ability to handle larger networks.

1.3 Contributions

Our study made several specific contributions to the field. First, our simulation tool will ensure that future research on bandwidth incentives can be conducted faster and easier. Through the utilization of concurrency and other implementation enhancements, we achieved a remarkable performance improvement in the new simulation tool, reducing the execution time needed by a factor of 10,000 compared to the original version. Secondly, we facilitate easier analysis of the different experiments conducted with the simulation tool due to our adjusted design. While our study may not have introduced many new or improved ideas regarding bandwidth incentives in Swarm, the adaptation and enhancement of the simulation tool hold great potential for aiding future research in this domain. By aligning the simulation tool more closely with the behavior of Swarm, we have created a valuable resource that can serve as a foundation for investigating and testing various bandwidth incentive mechanisms.

1.4 Outline

There are eight chapters in this paper:

- Chapter 1 serves as an introduction to our paper and work.
- Chapter 2 aims to provide relevant background information.
- Chapter 3 describes the design of the original Python simulation tool.
- Chapter 4 discusses different alternative designs to facilitate concurrency before detailing the final design of the new simulation tool.
- Chapter 5 delves into the implementation enhancements and specific changes to better align the simulation tool with the behavior of Swarm.
- Chapter 6 evaluates the adjusted design and performances enhancements.
- Chapter 7 discusses our work.
- Chapter 8 provides the final conclusion of our work and result.

Chapter 2

Background

The advent of the Internet has revolutionized the dissemination of information, significantly reducing the costs associated with it. With the emergence of the World Wide Web (WWW), every user now possesses the ability to be a publisher. However, it is important to recognize that while the costs have been greatly diminished, they are not entirely eliminated and how these costs are allocated plays a pivotal role in determining who can publish content and who can access it.

To gain a comprehensive understanding of the importance of this paper and the significance of the simulation tool, it is crucial to delve into the realm of decentralized storage networks, before focusing specifically on Ethereum Swarm. Additionally, we must explore the incentives driving the development of Swarm and identify the key features of Go that can aid in enhancing the Python simulation.

Decentralized storage refers to a network architecture where data is distributed across multiple interconnected computers, also known as peers or nodes. These nodes are part of a peer-to-peer (P2P) network, working collaboratively to manage and store data. Data uploaded to the network gets split into small pieces called chunks and gets sent to multiple nodes across the network for storage. The retrieval process in a decentralized storage network involves gathering the sharded components of data from the individual nodes that are responsible for storing them and subsequently reassembling them. Resulting in no single point of failure, in contrast to relying on a single centralized entity for data management and storage [4]. As well as making it significantly more challenging to tamper with or lose data, thereby enhancing its security and reliability [9]. The use of multiple

nodes in the network also provides a greater pool of available storage resources. Finally, the cost of utilizing decentralized storage is lower compared to existing centralized storage cloud platforms [9].

While decentralized storage systems offer many benefits, there are still some limitations to be aware of. Due to the decentralized nature of the system, the access times for storing and retrieving data may be slower compared to centralized systems. In addition, network outages could potentially impact the availability of the data stored [4].

2.1 The Ethereum Swarm Network

Swarm [24] is an open-source, decentralized p2p network built on the Ethereum blockchain. Swarm's primary objective is to act as a storage backbone and virtual server for a wide range of decentralized applications and content. It aims to maintain censorship resistance and protect the privacy of the network's users [18]. To ensure its sustainability, the Swarm network provides an incentive mechanism based on tokens to encourage contributions of bandwidth and storage from its members. The ultimate goal is to create an economically viable ecosystem for all participants involved [16].

2.1.1 Swarm data sharing steps

To facilitate a better understanding of the fundamentals involved in Ethereum Swarm [24], we have summarised it into four key points:

1. **Content Upload:** The first step in using Ethereum Swarm is to upload content to the network. This can be done by creating a manifest file that contains information about the content, such as its size, type, and location.
2. **Content Distribution:** Once the content has been uploaded, Ethereum Swarm automatically distributes it across the network. The content is broken down into smaller pieces called chunks, which are then replicated and stored on multiple nodes in the network.
3. **Retrieval:** When someone wants to access the content, they can request it from any node in the network. That node will then continue the routing pro-

cess to retrieve the necessary chunks and assemble them into the original content.

4. Incentives: To encourage nodes to participate in content distribution, Ethereum Swarm uses a system of incentives. Nodes are rewarded with cryptocurrency tokens for storing and serving content to others in the network.

2.1.2 Routing in Swarm

The goal of Swarm nodes is to establish and sustain a semi-permanent connection with other nodes in the network using an overlay addressing scheme. Due to the fact that nodes operate independently as autonomous agents, a distributed hash table (DHT), also known as a routing table, is employed. The routing tables serve as a crucial component responsible for managing and organizing the connections between nodes. For its scalability properties, Swarm uses Forwarding Kademia [19] as its coordinator service. By following a specific connectivity pattern, it is ensured that every node in the network can be reached within a certain maximum number of hops (which is logarithmic to the size of the network) when nodes exchange messages. Swarm's content, consisting of fixed-size 4KB data chunks, shares the same address space as the nodes in the network [24]. As chunks are uploaded to the Swarm network, they are gradually propagated from one node to another, with the objective of ultimately storing the data at nodes whose addresses are closest to that of the chunks address [18].

In the Swarm network, nodes employ the XOR metric to assess the proximity between chunks and other nodes, enabling precise distance calculations within the context of the address space. The XOR metric involves performing a 'bit-wise exclusive' operation between two network addresses represented in their binary format, which reveals their distance from each other. Based on these binary values, each node creates a routing table organized in a binary tree structure. Each level of the tree corresponds to a different bit position in the binary values [19]. The utilization of a binary tree structure facilitates efficient and scalable routing of messages between nodes within the network.

Figure 2.1 illustrates an example of a routing table in Swarm. In this particular example, the address size is three bits, and the routing table is employed by the node with the address 110. The routing table comprises three bins, each corresponding to a specific bit in the address size. These bins consist of lists of nodes,

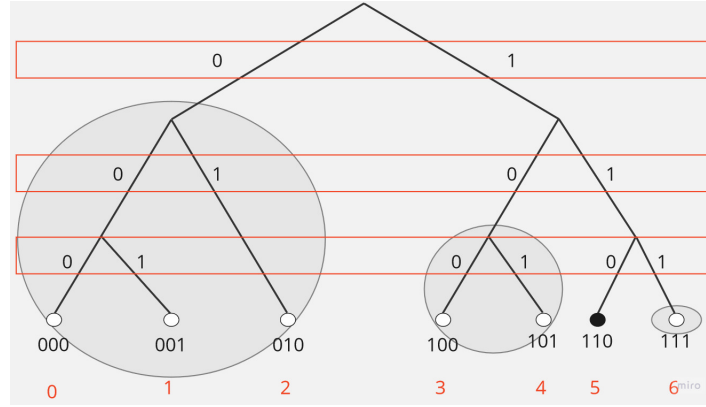


Figure 2.1: Routing table for node 110 organized in a binary tree structure.

corresponding to different parts of the network. The nodes that can be included in each bin are determined by the number of matching bits in their addresses. The illustration visually represents the network parts associated with each bin, indicated by circles surrounding the node with address 110. For instance, the first bin includes nodes that have a different first bit than node 110, which is showcased by the biggest circle that encompasses half of the network. Similarly, the second bin includes nodes that have a matching first bit but a differing second bit, which encompasses half of the remaining network. The third and final bin includes only the node with address 111, as there is only one alternative due to the address size being three bits.

With this routing table and the addition of nodes from each part of the network illustrated by the circles in each bin, the node with address 110 has the ability to reach any part of the network by contacting a node in the appropriate bin. Additionally, it is noticeable that the closer the nodes are to the requesting node, the fewer nodes are present in each circle. This implies that there is a greater chance of finding the desired chunk within each circle, or more specifically each bin, as the distance between nodes decreases.

The routing tables in Swarm are designed to give nodes a comprehensive understanding of the address space closest to them, with exponentially decreasing knowledge of more distant address space [19]. As a result, when a node wants to send a message to another node, it can efficiently traverse the binary tree structure by following the path that corresponds to the XOR distance between the source and destination nodes. This allows messages to be routed accurately and

efficiently in a decentralized network like Swarm [16].

In practice, the routing logic follows the following procedure. A node uses the XOR metric on its own address and the address of the data chunk within the network and calculates the distance. The node then inspects its routing table and searches the bin corresponding to the correct number of left-matching bits derived from the XOR calculation. It then looks for subsequent matching bits in the bin to decide which node is the closest to the destination. After identifying the nearest node to the destination, the node forwards the request for the data to that node. The responding node then follows the same logic by examining its own unique routing table to locate the next nearest node to the destination. If the node finds another node that is closer to the destination than itself, it forwards the request to that node. This same procedure occurs at each hop, continuing until the request reaches its final destination, at which point the data gets returned following the same route in reverse.

The described routing scheme is called forwarding Kademlia and is shown in figure 2.2.

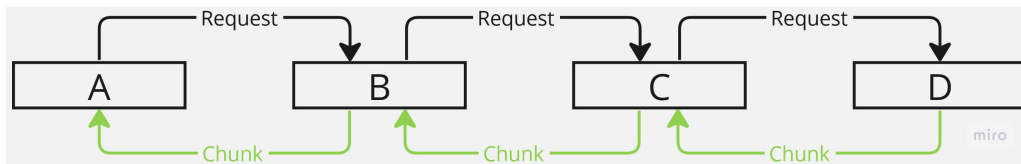


Figure 2.2: Forwarding Kademlia. A reaches D using forwarding Kademlia. In the route, a node is solely informed about the preceding node.

It's worth noting that the routing scheme forwarding Kademlia is different from the original Kademlia scheme [19]. During the lookup procedure in Kademlia shown in Figure 2.3, the requesting node contacts other nodes repeatedly to obtain either the chunk or addresses closer to the chunk. This process allows all the participating nodes to learn the identity of the requester [18], which goes against the philosophy of Swarm [24].

2.1.3 Incentives driving swarm

As previously mentioned Swarm has a rather complex incentivization model. Research by Lakhani et al [18] has identified multiple reasons why incentives play a critical role in the network:

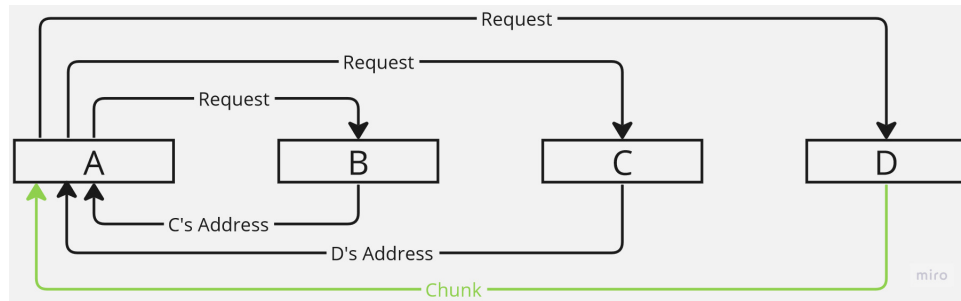


Figure 2.3: Kademlia. A reaches D using Kademlia by asking B, then C. When using Kademlia, all nodes that have been contacted are aware of the requesting node (originator) that initiated the request.

- To motivate nodes to join the network through the expectation of a monetary profit.
- To ensure that the behavior of each node is in line with the best interest of the network.
- To make deliberate attacks on the network costly to mount and persist.
- To make the initial joining to the network easy and allow for some free riding before needing to pay, or alternatively settle for reduced bandwidth.

The main focus of the Python simulation tool is to explore the positive incentives (rewards) for data transfer (bandwidth sharing) [16]. Hence, it is essential to understand the key components of Swarm's bandwidth system.

The Swarm Accounting Protocol (SWAP), shown in Figure 2.4, is the central component of Swarm's bandwidth incentive system [24]. Accounting units are used to enable nodes to monitor the amount of bandwidth service they provide and consume from connected peers. This information is recorded to track the relative bandwidth service provided and consumed [25]. This means that if a chunk is delivered to a node, the node keeps track of that it owes something to the node which provided the service [14].

The cost of each upload and download request is determined by the XOR distance between the requester and the requested chunk. This means that the closer a node is to the chunk, the lower the price is to forward the request. The algorithm used in Swarm[24] ensures that every next node in the route is closer than

the previous one. As a result, each node in the route pays less to forward the request for the chunk than what it has received by answering the previous node's forwarding request. This ultimately allows each node in the route to make a profit, with the exception of the original requester. Because of this, nodes are motivated to help find chunks [14]. In Swarm [24], node pairs have the ability to determine the maximum allowable debt between them, which is commonly referred to as the threshold limit. And in the event this threshold would be exceeded, part of the debt needs to be settled before further forwarding requests will be accommodated.

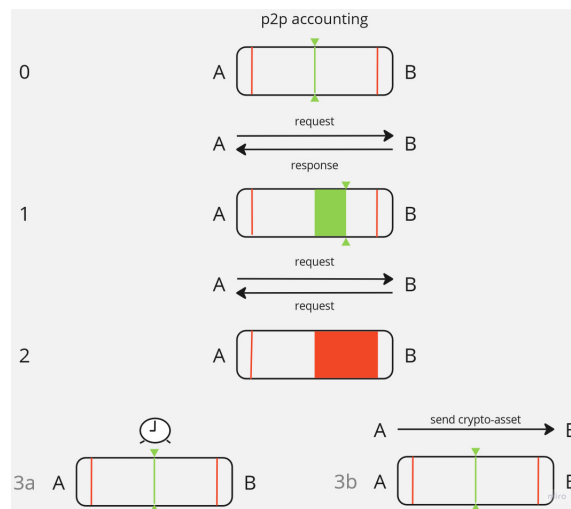


Figure 2.4: p2p accounting protocol. The figure shows how peers maintain track of their bandwidth contributions. The process commences with a zero balance on both sides at step 0, followed by a series of message exchanges that advance from step 1 to step n, ultimately resulting in one side's debt reaching a predetermined threshold. The debtor can choose to amortize the debt incrementally over time, or they can offer compensation to the creditor.

To settle the debt between node pairs in Swarm, there are two approaches. The first approach is to pay the amount owed by the node to the other node. Alternatively, since balances are gradually reduced over time, there is a continuous movement toward zero debt. In this case, the indebted node can wait until the debt is sufficiently reduced before requesting more data. Hence, as a feature in Swarm, nodes have the option to offer a limited amount of bandwidth per unit of time and connection without charging any fees. By offering this feature, anyone

can request content from Swarm for free, although at a slower rate [18]. This can encourage greater adoption of the platform.

2.2 Go vs Python

Our new simulation tool is built upon an existing simulation tool, which is implemented in Python. Therefore it is important to understand the key differences between Python and Go. Although both languages are widely used, they exhibit fundamental differences:

- Go's syntax is designed to be simple and easy to read, with a focus on minimalism and efficiency [8]. Python, on the other hand, prioritizes readability and ease of use, with a syntax that is designed to be intuitive and straightforward [11].
- Performance: Go is a compiled language, which means that it generally performs better than Python, which is an interpreted language [11]. This means that Go programs are generally faster and more efficient, especially for tasks that involve heavy computation or data processing. Previous research has concluded that Go can perform significantly faster than Python, with studies suggesting that it can be between 30 [20] to 40 [22] times faster.
- Concurrency: Go is designed with concurrency in mind, making it easy to write programs that can handle multiple tasks simultaneously [8]. Python also supports concurrency, but it requires more effort and expertise to write concurrent code that performs well, as well as only running on a single CPU core [11].
- Libraries: Python has a vast library of third-party modules that can be easily installed and used for a wide range of tasks [11]. Go, on the other hand, has a more limited library, but it has built-in support for common tasks like networking and files I/O [8].
- Community: Python has a much larger and more established community than Go, which means that there are more resources, tools, and support available for Python developers [11]. Go, however, has a growing and enthusiastic community, with a focus on developing high-performance, scalable applications[8].

In summary, Go and Python is both powerful languages that can be used for a variety of tasks. While Go is faster and more efficient, Python is easier to use and has a larger community and library of resources. The choice between the two largely depends on the specific needs of the project and the expertise of the developers.

2.3 Concurrency in Go

Go is a programming language that has been specifically designed to support concurrency [13]. In this section, we will introduce some of the fundamental concurrency primitives in Go. Additionally, we will provide an overview of the producer-consumer pattern, a widely used pattern in concurrent programming.

2.3.1 Go Concurrency Primitives

Go has access to traditional concurrency primitives such as mutex locks, but it also introduces other primitives that have helped make it a popular choice for programs that heavily depend on concurrency. According to Donovan and Kernighan [8], the points mentioned below are a summary of the most important primitives in Go:

- **Goroutines:** Goroutines are lightweight threads managed by the Go runtime. They can be thought of as functions that run concurrently with other goroutines. By having multiple goroutines running different functions, or the same functions in parallel, the performance can be sped up dramatically [8]. However, this also brings in true randomness and the potential for data race conditions, which will need to be accounted for [8].
- **Mutexes:** Mutexes are used to protect shared resources from concurrent access by multiple goroutines. Mutexes can be locked and unlocked using the `Lock` and `Unlock` methods. They provide a means to guarantee that only one goroutine can access a shared resource, or the section of code encapsulated within the `Lock` to `Unlock` span. Once a goroutine acquires the mutex, any other goroutines that require the same mutex will be forced to wait until the holder releases it and it becomes available again [8].

- **Atomic Package:** The atomic values and atomic functions offered in this package are used for low-level synchronization and thread safety. Atomic operations are used to ensure that multiple goroutines can safely access and modify a shared variable without causing race conditions or other synchronization issues. In practice, it can be used similarly to a Mutex for a single resource, just with the locking and unlocking logic abstracted away [8].
- **Channels:** Channels provide a way for goroutines to communicate with each other and synchronize their execution. They have typed conduits that allow sending and receiving values between goroutines. Channels are made to be thread-safe, so it's often used in Go compared to other languages which have to rely on alternatives, such as global variables with mutex locks. They also have the advantage to have buffers, where values can be stored in the channel temporarily before another goroutine retrieves them [8].

These primitives allow for safe and efficient usage of concurrency in Go programs.

2.3.2 Producer-Consumer Pattern

The classic synchronization problem of the producer-consumer pattern involves the division of labor between threads. This pattern is frequently utilized in multithreaded applications that require data-intensive tooling, particularly in tools used for ingesting and processing large amounts of data [10].

Figure 2.5 illustrates two pools of threads: producers and consumers, and each thread has access to a shared queue. Producers are responsible for generating or producing items, while consumers are responsible for processing them [10]. Typically, the threads assigned to each pool in a producer-consumer pattern can be separate, but the underlying logic may remain the same.

Upon reading data, a producer thread adds it to the shared queue, and consumer threads retrieve data from the queue to process it, potentially yielding the data for additional actions or storage [10]. This decouples the producer and consumer, enabling them to work independently and giving control over how many threads at a time are engaged in detecting or executing work. Compared to creating individual threads per item of work, utilizing a set consumer group has advantages worth considering.

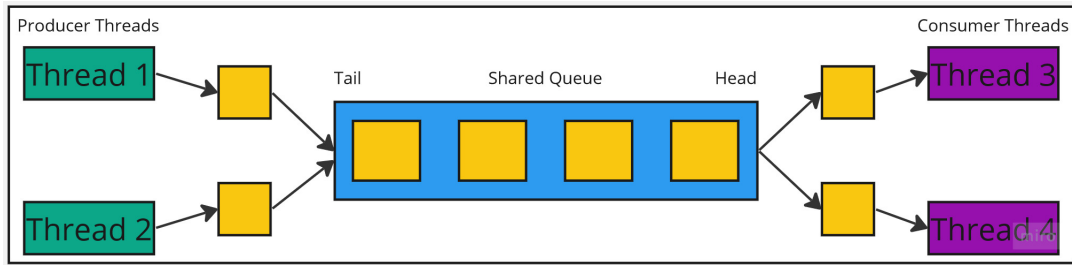


Figure 2.5: Producer-Consumer Pattern

Firstly, the overhead associated with thread creation and management is significantly reduced by using a fixed consumer group. Creating new threads for every piece of work adds extra computational cost while also causing potential issues like resource contentions or synchronization problems within the overall system structure [27].

Lastly, working within this pattern makes it easier to maintain overall stability in terms of resource usage, as handling concurrency levels is simpler when dealing with only established users involved in the process [27].

Chapter 3

Original Design

The purpose of developing the original Python simulation tool, which will from now on be referred to as PySwarmSim, was to investigate and gain insights into the bandwidth incentives of Swarm [24]. As the simulation tool kept its focus on specific aspects, some essential features of Swarm were intentionally simplified or omitted. This chapter will look into how PySwarmSim was designed, and walkthrough its logical steps.

3.1 Design With CadCAD

PySwarmSim simulation tool was developed using the open-source simulation engine cadCad (Complex Adaptive Dynamics through Agent-based models) [12]. CadCAD is a widely recognized open-source toolkit utilized by the interdisciplinary token engineering community. It is frequently employed in designing token economics due to its ability to simplify the development, testing, and validation of complex systems. The aim of the tool is to improve the understanding of payment models in p2p networks at large by focusing on Swarm [24] and assessing the fairness of algorithms with different parameter settings [16].

To enable modularity and flexibility in modeling complex systems, CadCAD follows a design principle with Policy functions and state Update functions, which involves separating the essential components of the simulation into two parts [7]. Policy functions define the decision-making logic or rules followed by agents or entities in the system. These functions determine the actions taken by the entities based on their states and the information available to them at each time step. As

such, Policy functions are responsible for generating the policies that govern the behavior of the system [7]. Update functions however define how the states of the entities or variables in the system change over time based on the policies defined by the policy functions. These functions take into account the current states of the entities and the actions determined by the policy functions to calculate the new states for the next time step. As such, Update functions are responsible for updating the system's state [7].

In PySwarmSim, CadCAD is responsible for initializing and running the simulation, and for handling the policy and update separation. The policy part in the context of the simulation means creating requests for chunks and then routing across the network for that chunk, which is done only by reading from the state and not doing any modifications to the data. The update part modifies the existing state based on the policy output, creating a new state that the next policy iteration will use, resulting in a self-feeding iterative loop.

3.2 PySwarmSim Simulation Steps

In this section, we will delve into the design of PySwarmSim and its logical steps during an iteration. The main focus of the simulation tool is to explore the positive incentives (rewards) for data transfer (bandwidth sharing) in Swarm [24]. Specifically the incentivization of forwarding chunk requests across the network to one of the nodes carrying the wanted chunk and then forwarding the requested chunk back to the client node (user). In order to simplify the simulation process, the tool does not incorporate the actual settlement mechanisms, and model the routing of data chunks as atomic operations, preventing any interleaving between two chunk retrievals. Although these assumptions may be unrealistic for individual requests, their impact on the overall system is minimal [16].

The simulation tool was designed using four general layers: the initialization layer, the policy layer, the update layer, and the post-processing layer [16]. Following the structure from cadCAD [12], the layers were designed to have separate and clear concerns to enhance the modularity within each layer, at the cost of having coupled logic between the interfaces of the Policy layer and Update layer.

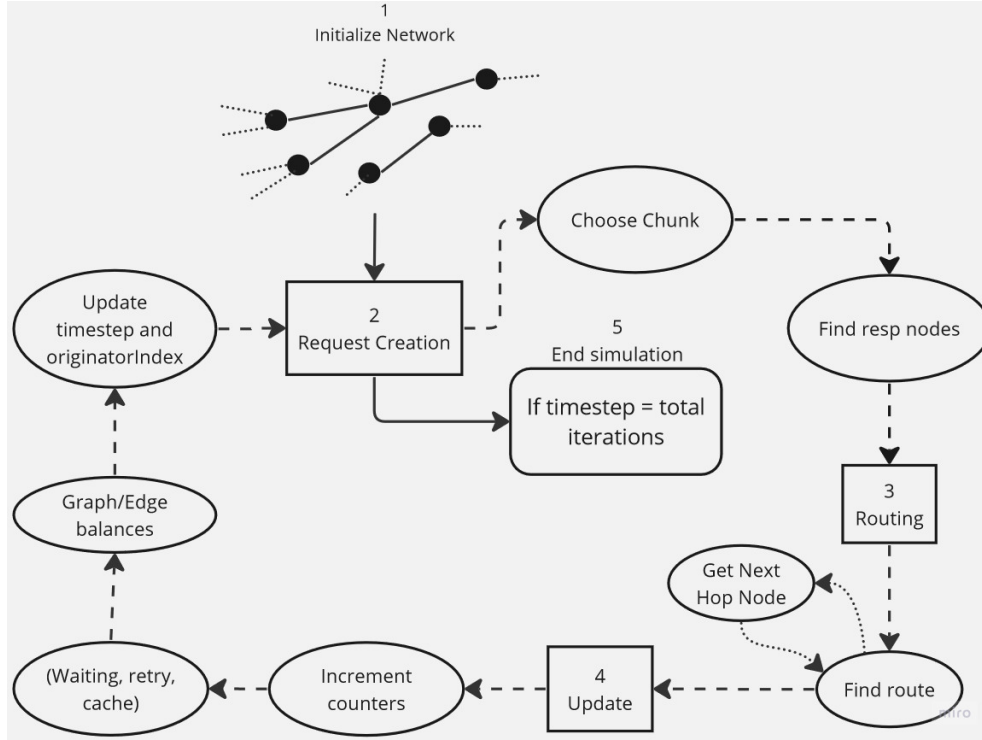


Figure 3.1: PySwarmSim simulation steps overview

Figure 3.1 illustrates the simulation steps of PySwarmSim and is a good overview of how the logic flow is designed. In the figure, step 1 is the initialization layer, steps 2 and 3 are the policy layer, step 4 is the update layer, and step 5 indicates the end of the simulation where post-processing can be done.

3.2.1 Initialize the Network

PySwarmSim starts every simulation run by initializing a network based on a generated file containing all the node addresses, and all the nodes in each other's adjacency lists, which is what PySwarmSim calls the bins from Swarm [24]. Similar to the bins, each adjacency list contains nodes in the network corresponding to the correct number of bits determined by the XOR calculation, allowing for efficient routing to every part of the network [19]. The number of adjacency lists contained by each node is the number of bits N in the network size, and the number of nodes in each list is the bin size K . The variables N and K are decided at the time of generation of the file, although the user can override the K value in

the configurations before a simulation run, with the result being less populated the adjacency lists.

Once the network has been created and kept in memory, a directed graph is constructed based on the connections between nodes present in the network. Specifically, for each node a directed edge from itself to every other node in its adjacency list is created, as well as an opposite edge connecting the other node back to the original node. The effectiveness of this initialization approach and the existence of two data structures with similar data can be argued, but that is the method utilized in PySwarmSim.

During the simulation run, the network with its nodes and their relationships stay the same throughout all the iterations. In contrast, the edge attributes of the edges in the graph are updated continuously during the simulation to reflect the balance between the nodes and any additional information useful for routing. One can therefore think of the network as the initial network state and the graph as the continuously changing state of the network. Because of cadCAD [12], the simulation has the option to save every one of those states, one for each timestep, and use that data for further analysis instead of running the simulation in full again.

Lastly, the initialization of the network chooses a number of the nodes as originators, nodes that will serve as the initiators of all subsequent requests during the simulation. The number of originators is decided by the user, but the subset of nodes chosen from the nodes pool is chosen at random. The list of originators is stored as a part of the global state, along with the state variable `originatorIndex` which is used to select an originator from the list. The value of `originatorIndex` is updated every iteration to determine the current active originator.

3.2.2 Request Creation

The second step in a simulation run, and the start of the main iteration loop, is the creation of requests. Every request starts by having an originator picked from the rotating pool of available originators. Then a chunk gets chosen, which is what the originator will want to be retrieved from the network. After which, the nodes responsible for carrying the chunk during the routing will be calculated.

In Swarm [24], the originator refers to the peer that initiates a request for a chunk, and it serves as the starting point for all network routing. In PySwarmSim,

this concept is simplified by designating the first node in a route as the originator. Although, being an originator does have some additional ramifications regarding edge balances and additional payments, which depend on the specific settings used in the simulation. The number of originators used during a simulation run can be specified by the user through the configuration settings. This allows the user to control the load distribution on the network and simulate different scenarios to analyze the network's behavior under various conditions.

As PySwarmSim is a simulation, the data distribution is done differently from Swarm [24]. Instead of having users upload data to the network and having the data be divided into chunks, which are then pushed toward the nodes in the network with an address most similar to the chunk's address. PySwarmSim simulates the retrieval of chunks in a simplified manner by having chunks just be an address in the address range.

As such, PySwarmSim has two methods for selecting the chunk that the originator will request. Normally this is done by randomly generating a chunk with its address at the time of the request creation. However, there is an alternative method used for handling routing failures, which involves retrying the routing for chunks that were not found in previous attempts. The process of selecting a chunk for retrieval operates in a hierarchical manner. If retries are enabled for the simulation run and an originator has failed to obtain previous chunks, these chunks are given higher priority than new requests. This functionality can be enabled or disabled based on the user's preference.

To keep track of the chunks that failed to be located during previous routing attempts, the chunks are stored into globally accessible objects depending on the configurations used. The first one is called a **pending_dict**, detailed in Section 3.2.4, which stores originator-chunk pairs. The second is called **reroute_dict**, detailed in Section 3.2.4, which stores originator-reroute pairs, where the reroute object contains a list of rejected first-hop nodes which the new routing attempts will skip, along with the corresponding chunk.

The final part of scheduling a request is to calculate the four nodes that can be responsible for the chunk. In PySwarmSim, nodes are not responsible for storing chunks themselves. Instead, the simulation calculates ad hoc which four nodes in the network have the closest XOR distance from the chunk address to the nodes' addresses. The act of successfully finding the chunk is determined by whether the routing reaches one of the four nodes responsible for carrying the chunk or

not, even though the nodes themselves have no knowledge of the chunk. This simplifies the simulation and allows for a faster and more efficient computation of the network's behavior. As a result, from now on, when we are talking about *finding the chunk* we actually mean finding one of the four responsible nodes determined to be closest to the chunk.

3.2.3 Find Routes

The third step in a simulation run is the routing. Once an originator requests a chunk to be retrieved from the network, the network in turn has to find a viable route to one of the nodes responsible for storing the chunk. The routing logic uses the data from the request, specifically the originator, the chunk address, and the responsible nodes as the input. Due to the significant variability in the combination of variables involved in each routing attempt, the routing logic can result in a wide range of possible routes and outcomes.

As the ultimate goal of the routing is to find one of the responsible nodes, the routing logic initially verifies whether the originator itself is among the four responsible nodes. If this condition is satisfied, then the originator already possesses the chunk, the request is deemed a success and the route is finished. If not, the main loop of routing starts, by continuously attempting to identify a next hop node that is closer to the chunk than the previous node and adding it to the route.

Starting the loop, the first step is to identify which node from the originator's adjacency lists can be added to the route. The simulation searches through all adjacency lists and compares every node to each other to find the one with the closest XOR distance to the chunk. When a node is found to be closer than the originator, the node also has to pass some additional conditions depending on the experiment chosen and the simulation configurations used to be considered a potential next-hop node.

In the simulation, if the threshold is enabled in the settings, the node must pass a threshold check. This check follows the same logic used in Swarm [24], which is explained in detail in Section 2.1.3. In this case, the simulation checks the balance on the edge connecting the originator, and the potential next-hop node. The goal of this check is to verify that the cost of requesting services from a potential next-hop node by the originator will not result in its edge balance exceeding the threshold limit. In the case that the edge debt would exceed the limit,

the potential node is considered invalid. However, if payment is also enabled in the simulation settings, then the originator in this case can simply pay the price the next-hop node requires, making the edge debt balanced again.

After successfully passing all necessary checks, the potential node is considered the current best candidate for the next-hop node. This process continues until no other valid candidate closer to the chunk is found, making it the optimal choice for the next-hop node. The selected node is then added to the route, becoming part of the path from the originator to the chunk. At this point, the process of finding another next-hop node restarts, but each time using the adjacency lists of the most recently added next-hop node.

There are a few ways this loop can eventually terminate. Firstly, the case when all the closer nodes in the currently last node's adjacency lists fail the threshold check. When this happens, the outcome of the routing is what's called a threshold failure. Secondly, in the case when no valid nodes in the currently last node's adjacency lists are closer to the chunk, the routing fails again. In this case, the routing outcome is what's called an access failure. Lastly, if the last node added to the route is identified as one of the four responsible nodes, the simulation considers the chunk found and the routing outcome is a success.

In the end, the outcome from the routing along with other results and output generated during the routing process (policy) is saved for later use as input in the update functions (update).

3.2.4 Update

The fourth step in a simulation run is the update layer, which involves updating parts of the global state of the network based on the output from the policy layer. After each scheduled request, a series of updating steps must be completed, regardless of the routing outcome. These updating steps will directly affect the outcome of the next scheduled request, as they alter the state of the simulation.

Increment Counters

When a scheduled request as explained in Section 3.2.2 is made, there are three possible outcomes from the routing: either the requested chunk is found, or one of the failures occurs. The global state keeps track of the number of routings that result in each possible outcome.

For this purpose, it maintains a counter for each threshold failure, access failure, and successfully found chunk. As such, in every update phase, the appropriate counter gets incremented matching the preceding routing result. After the simulation has finished, the user can view these results to get an overview of how the simulation performed.

Update Originator Index

The scheduling process starts with picking an originator from the rotating pool of available originators based on the state variable `originatorIndex` that's stored in the global state. The originator index starts at a value of **0** and is incremented by one after each request during this phase.

Since there always is a predetermined number of originators available, the originator index is set back to **0** once it has reached the end of the list containing the originators. This ensures that the requests are evenly distributed among all the originators and that each originator has a fair chance of processing/scheduling requests.

Update Pending Dict

The global pending dictionary updates only when the 'waiting is enabled' configuration option is on. It contains a key-value pair that connects an originator to a chunk, which is created when a routing for a request fails. Subsequent requests from the same originator will prioritize the chunk from the dictionary, and route for the chunk once more. Since routings with the same originator, chunk, and responsible nodes have the same input, the route across the network will stay the same. Therefore, if a previous routing failed, and the failure was due to a threshold limit issue, it is possible that the issue has been resolved in the following iterations. For example, if the failed edge has had some of its debt forgiven, the simulation may now be able to find a successful route.

The originator key and chunk are removed from the dictionary only when the chunk is eventually found. This means the originator will keep on requesting the same chunk, even if it is not possible to be found based on the input variables. If however a successful route is found, the originator key and chunk are removed from the dictionary, and subsequent requests from the same originator will be for a new chunk rather than the previously failed one.

Update Reroute Dict

The global reroute dictionary updates only when the 'retry with another peer' configuration option is on. It contains a key-value pair that connects an originator to a reroute list, which is created when a request for a chunk fails due to a threshold failure or an access failure. The reroute list includes the first-hop nodes from the originator along with the requested chunk and gets updated with each failed request from the same originator. Subsequent routings from the originator use the reroute list for the retries, to prevent revisiting previously rejected first-hop nodes, increasing the likelihood of finding a new successful path to the chunk.

The originator key and reroute list are removed from the reroute dictionary in two cases: when a successful route to the requested chunk is found or when the reroute list exceeds the bin size. At the point of the latter, it is deemed unlikely to find a successful route, and little reason to continue looking as every node in one of the originator's adjacency lists has been checked. In either case, the deletion ensures that the global reroute dictionary remains up-to-date and efficient, and the subsequent request coming from the same originator will be for a new chunk.

Increment Timestep

Similar to the counters used for the routing results, PySwarmSim employs a timestep counter as part of the global state, which gets incremented during every update phase. The timestep is used to keep track of how many iterations have been completed, needed to be able to decide when the simulation is finished, but also has additional use cases.

For instance, regarding the concept of how many seconds have passed, where a second is deemed to pass every 12,500 timestep. Certain aspects of the logic are only applicable when a new second begins. For instance, the amortization of edge debt with the forgiveness logic as explained in the following Section 3.2.4, is only valid after waiting for 12,500 timesteps.

For instance, time is measured in terms of the number of timesteps that have passed since the start of the simulation. A timestep represents a fixed amount of simulated time, which in PySwarmSim is set to 12,500 timesteps per second. Additionally, certain aspects of the simulation logic are only applicable when a new second begins, such as the amortization of edge debt with the forgiveness logic, which is explained in Section 3.2.4.

Update Balances on Edges

Besides scheduling requests and finding routes to the chunk across the network, updating the edges in the graph every iteration is also crucial to properly reflect how Swarm [24] handles the bandwidth incentives. The directed edges in the graph have several attributes, including the edge debt from one node to another. Additionally, the timestep when the edge's debt was partly forgiven last, along with an optional dynamic threshold limit.

The initial stage of updating the balances involves applying the payments included in the routing output, which is only relevant when payment is enabled in the simulation settings. If an edge is part of the payment, we assume that the requester has the capability to pay. In this case, the payment amount is equal to the calculated price minus the debt the responder already owes. Both directed edge pairs will then have their debts reset to zero. The payment system is used to reflect how in real Swarm [24], a frequent downloader can pay in order to continue requesting more data.

The next step is to calculate the price for every hop in the route and then add it to the corresponding edge debt, to calculate the final profit for each node. This is only done when the routing was successful so routings that end in an access failure or threshold failure don't count towards adding debt to edges. By doing it this way, the originator doesn't have to pay for a chunk that wasn't found, but we still end up wasting some of the bandwidth between the nodes with no value given to the responders.

The following step involves determining the price for each hop in the route and then adding it to the corresponding edge debt to calculate the final profit for each node. This calculation is only performed when routing is successful, where failed routes do not contribute to the addition of debt to edges. This approach ensures that the originator is not charged for a chunk that was not found, but in the context of a simulation, some bandwidth between nodes will go to waste with no value provided to the other nodes in the route.

Lastly, all the edges between two nodes that potentially had their threshold limit exceeded during routing are checked to see if they fulfil the forgiveness criteria. When the time steps since an edge was forgiven last is at least one second or 12,500 iterations, a portion of its debt is forgiven based on the duration of time that has passed. The purpose of this feature in Swarm [24] is to simulate the

amortization aspect of the accounting, which allows for some free-riding for infrequent downloaders and users who do not wish to participate in the payment system.

When all of these logical steps are gone through after every routing, which directly affects the next routing, the network with the edge balances and payments are kept consistent and fair.

3.2.5 Extract and Analyze Data

After processing a request, the simulation updates various variables and generates an output that describes the route created by the request. This output is displayed in the terminal and includes information such as the originator of the request, the nodes used in the route, and the payment made by each node for each hop. This output can be examined to learn more about how rewards work in a peer-to-peer network, and different settings can be tested to observe their effects. The output can be saved to a file via terminal redirection to make data analysis easier.

Every route that is traversed in the simulation is also stored in memory. To reduce memory usage this is extracted and dumped to a file after a set amount of iterations. The output file then holds every route of the simulation which also can be used at a later time for further analysis.

3.3 Conclusion on Original Design

In conclusion, PySwarmSim proves to be a valuable simulation tool, offering a user-friendly interface and enhanced insights into the effects and mechanisms of token-based incentives. While the design approach and underlying assumptions of PySwarmSim are logical, it is apparent that the tool prioritized simulation results over performance. Therefore, there is room for improvement in terms of its efficiency, speed, and overall quality. Nevertheless, this presents an opportunity for further development and enhancement of PySwarmSim's capabilities.

Chapter 4

Adjusted Design

Our new simulation tool, which will from now on be referred to as GoSwarmSim, builds upon the foundation established by the PySwarmSim simulation tool, while also incorporating several key enhancements and modifications to the design. Throughout the development process, we explored various design alternatives, ultimately settling on a single design we deemed the most optimal. In the upcoming sections, we will explore the changes made to our design to replicate cadCAD’s functionality. We will discuss our approach to implementing concurrency and introducing goroutine workers, as well as integrating the concept of epochs. Finally, we will present the resulting adjusted design.

4.1 Design Without cadCAD

The PySwarmSim was created with the Python package cadCAD [7] serving as the orchestrator behind the simulation. Therefore, we must explore alternative approaches to achieve similar simulation functionality in GoSwarmSim. That is, how to coalesce all the different logical functions into a working self-feeding iterative loop, similar to the Policy and Update design from cadCAD.

Initially, cadCAD seemed like a mysterious black box that produced results without a clear chain of events and no obvious way to see how all the different logical pieces fit together. However, after studying the framework more closely, we realized that its complexity stems from its flexibility to simulate a wide range of scenarios, which isn’t necessary for our purpose.

4.1.1 Replicating the needed functionality from cadCAD

Our first approach for an adjusted design was replicating the orchestrating utility of cadCAD with a simple main loop while keeping the policy and update separation. The first step involves initializing the network and its nodes together with the graph and its edges as normal, creating an initial state serving as the starting point of the main loop. Each subsequent iteration of the main loop has its own modified version of the state, a current state. The design then generates a request for a chunk based on the current state, determines a route through the network for that chunk, and then updates the state based on the results, creating a new current state.

By utilizing a compiled language like Go instead of Python, the performance benefits of this simple approach are significant, resulting in the same expected output as PySwarmSim but with significantly less execution time. However, as the design is almost identical to the PySwarmSim, which needed to adhere to the cadCAD method of doing things, it is likely that a more optimal approach exists. Keeping the design so similar, and having the coupled separation from policy and update, will limit our ability to make design changes and optimizations, and add additional new features.

4.1.2 New design without cadCAD's limitations

Preserving the resemblance to the design of PySwarmSim and avoiding excessive divergence in terms of the simulation steps was initially a logical choice. As a result, the starting point of our designs mimics the design illustrated in Figure 3.1 in Section 3.2. Although we use the same logic as depicted in Figure 3.1, our implementation is different because of the lack of cadCAD, which affects how the logical parts work and how they are connected. This strategy provides a foundation for GoSwarmSim to evolve from and also enables straightforward comparison between the two simulation tools.

Illustrated in Figure 4.1 we determined the main loop can essentially be split into three parts, by splitting the policy layer from PySwarmSim:

1. Creation of requests, choosing a chunk, and finding the responsible nodes.
2. Routing across the network to find one of those responsible nodes, which in turn means finding the chunk.

3. Updating the global state based on the output from the routing.

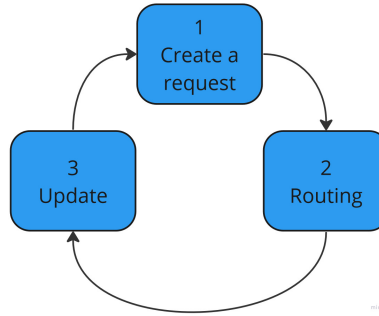


Figure 4.1: Illustration of the main loop

As stated earlier, we wanted to decouple the policy and update parts in order to redesign the simulation tool to achieve one of our main objectives, which is the implementation of concurrency. The main issue being the updating of the network must occur after every request and routing to ensure that the subsequent request and routing use the correct data. This poses a challenge when incorporating concurrency into our design without introducing data inconsistencies or race conditions.

Nevertheless, the main loop with the lack of cadCAD provides freedom to introduce changes and move logic from one part to another, make optimizations, and add new features in each part separately. This allows for a more refined and efficient implementation of the simulation, making the introduction of concurrency possible and resulting in an overall performance improvement.

4.2 Adjusting the design to implement concurrency

As described in Section 1.1, our main objective with this project is to utilize the features of the Go programming language to make improvements to an existing simulation tool. One of Go's major features is the built-in support for concurrency [8]. Therefore, one of the key objectives was to incorporate concurrent programming techniques into our design.

To implement concurrency in our design, we have explored three approaches. Each approach focuses on specific parts of the main loop illustrated in Figure 4.1. The first approach utilizes multiple goroutines to concurrently explore different

paths across the network for the requested chunk during routing. In the second approach, we merged the request and routing parts and executed them concurrently, while keeping the updating part separate. Lastly, the third approach introduces a global lock to ensure thread safety in the updating process.

4.2.1 Concurrency inside routing

The initial attempt to incorporate concurrency was to concurrently traverse different paths across the network. By using concurrency inside the routing as seen in Figure 4.2, the goal was to increase the chance of finding a valid route to the chunk, while also producing a result quicker and more efficiently. In order to achieve this, at the start of each routing attempt, k goroutines were started, identical to the bin size. Each goroutine was assigned one of the originator's adjacency lists, where each goroutine would start the routing process by selecting a separate first-hop node according to the nodes in its assigned list. By having every goroutine get a different first hop node, GoSwarmSim would produce k distinct routes, increasing the likelihood of finding the chunk in one of them. The first goroutine to find the chunk would return the route with the chunk, resulting in all the other goroutines being terminated to not waste resources.

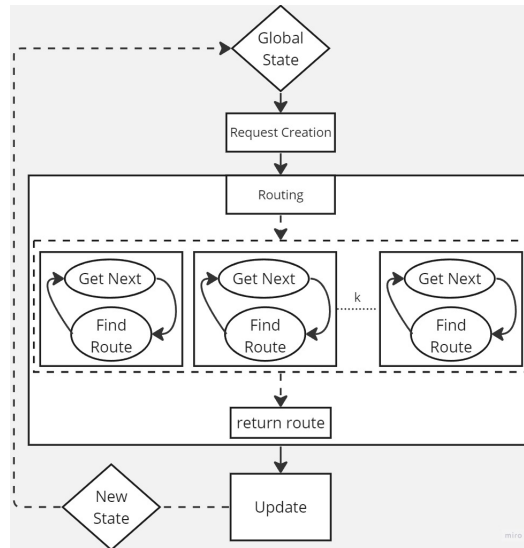


Figure 4.2: Concurrency inside routing. Multiple goroutines are generated to find a valid route to the requested chunk. The first route that successfully finds the chunk will be selected.

However, we discovered that starting and suspending k goroutines for every routing attempt to locate a single valid route can be resource-intensive, particularly when the workload assigned to the goroutines is minimal. Additionally, since the goroutines were initiated simultaneously, most of them finished their routes at the same time. Consequently, despite the increased likelihood of finding the chunk, the overhead introduced by continuously starting and suspending multiple goroutines resulted in a decline in GoSwarmSim's overall performance [27].

4.2.2 Concurrent request and routing - Naive approach

The second design change was merging the request and routing parts and executing them concurrently while the updating part was kept separate as illustrated in Figure 4.3. The new combined part, resembling the policy layer from PySwarmSim can execute an arbitrary number of chunk retrievals concurrently.

During the simulation when a goroutine finishes a routing, it places its output into the list containing all results from the goroutines. After each goroutine is finished, the update logic is executed on the network by iterating through each of the results in the list one at a time. Once all the updates were completed, a new global state was established, which will then be utilized by the next iteration of request creations and routings.

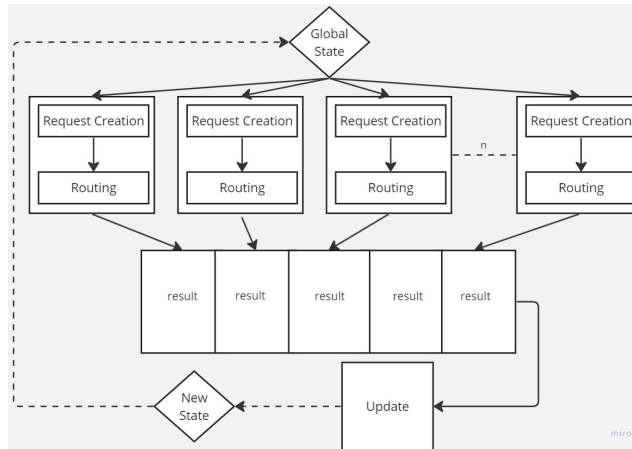


Figure 4.3: The request creation and finding the route happens concurrently through n goroutines. The output of each goroutine is then forwarded to the updating component to modify the network's global state.

This approach, as it turns out, is rather naive and ultimately yields an end result that deviates significantly from the expected results. While updates occur in sequence, preventing any data races on the state objects, the problem lies in the results produced by the routings. The fact that all concurrent chunk retrievals refer to the same unchanging global state and any routing results that should be reflected in the current state will not be updated until the following iteration.

For instance, the routing that completes last within an iteration will use the same data as the first routing with regards to edge balances, which can cause issues if the edge already has been used by another goroutine. Suppose that the first routing had an edge that just barely passed the threshold check, then a subsequent routing that tries to use the same edge should fail the threshold check. However, since the edge balances remain unchanged, at least until the update step before the next iteration, both routings will succeed.

Our efforts to incorporate concurrency in our design were somewhat successful, as we allowed for concurrent request creation and routing of chunks, resulting in substantially improved performance. We found that the reduction of execution times was proportional to the number of goroutines employed, with each increment to the total number of goroutines resulting in faster execution times. However, as we discovered, the design alternative could not be utilized further due to the incorrect results it produced. Despite this, we noted the significant performance improvement achieved by increasing the number of goroutines, which remained an important consideration when exploring subsequent approaches.

4.2.3 Concurrent request and routing - Global lock approach

To rectify the flawed design discussed in the previous section, the global state of the network must be updated with the routing results after each routing, to ensure the consistency of subsequent routings. This means that we should combine all the parts in the main loop iteration into a combined block logic, and then run through the entire logic concurrently. This will allow the goroutines to execute the update logic themselves based on their own routing results after completing a routing attempt for a chunk.

To handle the possible data races to the state objects which the concurrent updating introduces, we incorporated a shared mutex lock [8] around the entire update logic. This means that only one goroutine can update at a time, causing

the others to queue up waiting their turn. Once a goroutine finishes updating, it is free to initiate a new request infidelity based on the new state it just helped create. Looking at Figure 4.4 we can see how these changes to the design are being done.

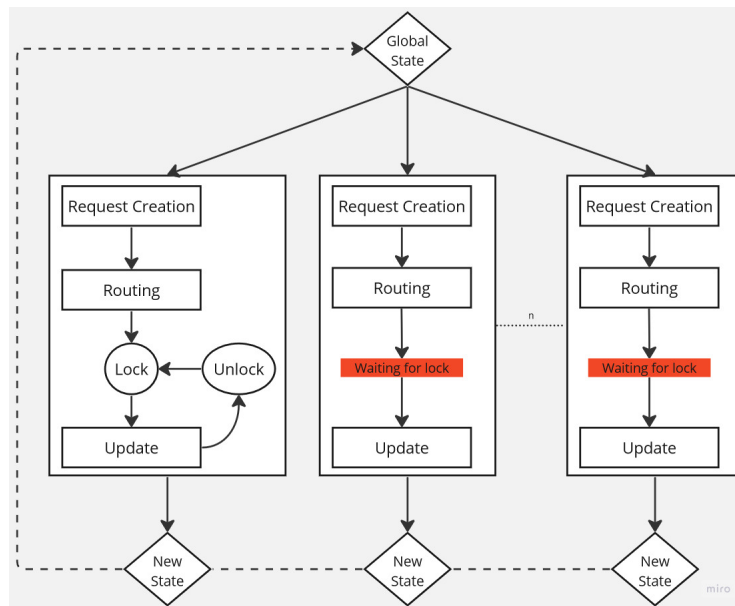


Figure 4.4: Concurrent request creation and route finding are facilitated through n goroutines, with each goroutine acquiring a lock to update the routing result.

The enhanced approach resulted in successful concurrency once more, while achieving the same expected end results that the original approach without concurrency produced, or at least similar enough when taking into account the inherent randomness that arises when introducing concurrency. While this is a valid approach compared to the naive design, the simulation is now being heavily bottlenecked by the update part with its mutex lock.

This means that there is a hard cap on the number of active goroutines, as increasing the number beyond a certain point will result in them waiting in the Mutex lock queue. We reach maximum performance for this design when the update logic is constantly being executed by one of the goroutines during the simulation, meaning we should also ensure that at least one goroutine is always waiting in the Mutex lock queue. After conducting some tests, we concluded that the sweet spot for the number of goroutines was around 10, where both increasing or decreasing the number beyond this point resulted in a deterioration of execution time.

To further increase the level of concurrency in our implementation and remove the bottleneck caused by the update logic, we need to analyze the logic inside the update part and identify the sources of data races and other concurrency issues that we previously experienced. Once we have identified these problematic areas, we can make them thread-safe individually by adding appropriate synchronization mechanisms to the resources used in those sections.

4.2.4 Making routing and update logic thread-safe

To ensure that the update part was thread-safe, making it possible to be executed by multiple goroutines concurrently, we had to identify which resources were susceptible to data races and other inconsistencies when executed concurrently.

The first challenge we tackled was when making the update of the counters for the different routing results, the timestep, and the value of the originator index thread-safe. Since these single integer values are shared resources and part of the global state, multiple concurrent goroutines need to be able to execute the associated increment functions whenever needed. To ensure thread safety, we introduced Go's atomic package [13] and exclusively used its atomic functions when reading or modifying the values. By doing so, we guaranteed that each function call to increment one of the values happened consistently, regardless of how many goroutines tried to increment a counter at the same time. The atomic functions are also faster than normal mutexes [5] because they use low-level CPU instructions to perform the atomic operations on shared memory without requiring expensive memory synchronization mechanisms. This means we already have fixed one of the issues when running the update logic concurrently.

The second challenge was making the global data structures for cache, waiting, and retry thread-safe when accessed concurrently by multiple goroutines both during routing and when updating. Initially, we attempted to solve this by using mutex locks to ensure that only one goroutine could access the global data structures at a time. However, we later found that this approach introduced significant overhead due to the goroutines waiting for the mutex locks [3], and therefore decided to move the data from global objects to objects on each node. By assigning each node its own set of mutex locks for the three data structures that were previously held globally, the need to wait for locks to become free was reduced or sometimes even eliminated. Despite the slightly increased memory usage caused

by the additional mutex locks, this approach simplified the mutex logic and improved performance, while also ensuring that the data structures were updated in a thread-safe manner.

The third and most significant challenge was ensuring thread safety during the graph update process, which involves updating the edge balances. Addressing this issue, we implemented mutex locks [8] for each edge in the graph to guarantee that multiple goroutines could safely access the edges and their attributes during routing. To avoid deadlocks [8], we made sure that both edges of a node pair shared the same mutex lock. During routing, a goroutine would now need to acquire the lock for the edge it wants to inspect, and subsequently release the lock if another closer node was found. When the closest node is found, the goroutine will hold the lock on the corresponding edge for the duration of the main loop until it reaches the update graph stage. After an edge is updated to reflect the result from routing, the corresponding edge lock is released along with the locks for all the other edges between the nodes in the route. This makes them available for use by other goroutines, thereby minimizing the waiting times [3]. By making these changes, we ensured that edge balance updates were done thread-safe, and avoided race conditions both during routing and when updating.

After implementing these changes, the update logic can now be executed concurrently without any risk of data races or other inconsistencies caused by concurrency. Although the changes increase complexity and memory usage, the benefits in terms of performance and scalability that come with running concurrent updates together with multiple concurrent routings for chunks far outweigh the drawbacks. The last remaining bottleneck is the edge locks, which can cause some waiting for other goroutines that want to access an edge already being used in a route by another goroutine. Although, when running the simulation without the edge locks activated, we see the performance difference is minimal.

4.3 Introducing Goroutine Workers

Building on the previous design choices with the addition of thread-safe update logic, for our final design approach, we decided to move the concurrency logic from anonymous goroutines to dedicated workers. Utilizing the producer-consumer pattern detailed in the background; Section 2.3.2, we split up the main iteration loop into a pool of a single producer and multiple consumers.

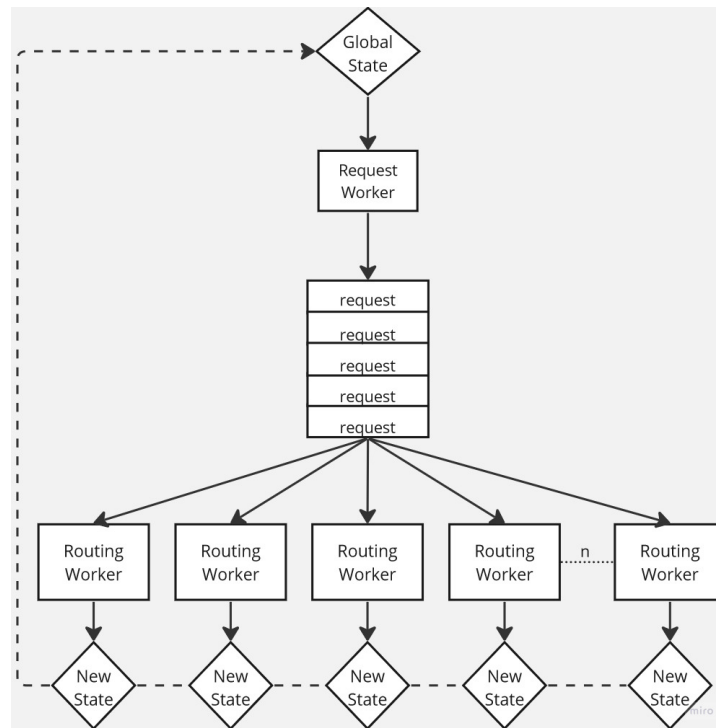


Figure 4.5: The RequestWorker generates requests that are consumed by the RoutingWorkers, which perform the routing and updating tasks.

This is showcased in Figure 4.5, where the single producer is the requestWorker along with the multiple consumers being the routingWorkers. The single requestWorker's responsibility in the final design is the continuous creation of requests and sending the request objects over a shared channel [8]. The routingWorkers will in turn continuously grab requests from the channel at their own leisure, before performing the appropriate routing and updating the current state based on the results from the routing. After the routingWorker has finished updating, it's ready to once more grab another request from the channel and start

its main loop again.

By using a producer-consumer design, each worker can function independently, thereby working at their own pace without interfering directly with one another. Furthermore, buffering the requests created by the requestWorker using a channel employs the same strategy. Consequently, it doesn't matter if the requestWorker momentarily generates requests slower than the routingWorkers can grab them as the buffer will compensate for this discrepancy.

The only potential bottleneck within this system would be an empty or fully buffered channel, both of which can be resolved by adjusting the buffer size accordingly or by scaling the relevant workers up or down.

4.3.1 RequestWorker

The requestWorker is a critical component of the final design, acting as a central reference point for crucial simulation details and serving as the main orchestrator for all other workers. As seen in Figure 4.5, everything flows out from the requestWorker, with the exception of references to the global state.

We determined that updating the timestep and originatorIndex values exclusively within the requestWorker would be more efficient than the previous methods of updating the values during the update section. This is because the requestWorker is the only worker using those values, rendering the previous approaches unnecessary. Furthermore, removing the need for atomic operations in the corresponding update functions simplifies the logic and slightly improves performance. Additionally, this way the requestWorker keeps track of the number of requests it has created and the number of iterations it has done, allowing it to in turn notify the other workers when the simulation has finished.

In the final design, every request object features a unique timestep, the originator, chunk address, and responsible nodes for that chunk to ensure proper routing for the routingWorkers. By leveraging the request object as a source of variables for efficient routing, the routingWorkers can also avoid unnecessary access to the global state, minimizing the risk of data races or other inconsistencies.

Similar to the logic in PySwarmSim detailed in Section 3.2.2, chunks associated with waiting and retrying for the originator will have a higher priority than newly generated chunks. Additionally, we have added an option to the configuration settings that can be enabled at will. When this option is turned on, if

the chunk selected for the request is a retry of a previously failed chunk, the request will not be counted toward the total number of iterations. As a result, the simulation will still generate N unique chunks, even when waiting and retrying are present, which helps to keep the results consistent. This topic will be further detailed in Section 5.7.

If none of those cases applies, the new request will generate a random address from the address range to be used for the new chunk. However, we have also added another option in the simulation settings which the user can enable. When enabled, the requestWorker will prefer certain chunk addresses in the address range, simulating a network where some data is more popular than others. This can be useful when running the simulation with cache enabled by getting the routingWorkers to often traverse the same routes increasing the cache hits or increasing the amount of possible threshold fails which could increase the chance for forgiveness, payments, or other measurable values.

The final task of the requestWorker is to determine the responsible nodes for the selected chunk. These nodes are essential for routing, as they are the nodes the routing needs to find in order to mark the routing as a success. Because of enhancements made detailed in Section 5.8, the calculation is less resource intensive compared to the PySwarmSim counterpart, usually only needing to reference a map with the chunk address as the key to get the four responsible nodes. Allowing the requestWorker to create requests quicker and minimizing the risk of becoming a bottleneck compared to the multiple routingWorkers expecting those requests.

In conclusion, the requestWorker is dedicated to solely creating requests and sending those over a channel with no thought as to how the data contained within is utilized. By having the coupled Policy and Update layers removed, we can create multiple requests sequentially without needing a routingWorker to finish updating to keep the state consistent, as the accesses to shared values on the global state are kept to a minimum. In fact, the data structures used by waiting and retry are the only shared resources the requestWorker references when creating requests.

4.3.2 RoutingWorkers

To simplify the process of determining the appropriate number of workers, we have a single pool of routingWorkers which will handle both the routing and updating for each request it grabs. Combined with the single requestWorker, this approach allows for concurrency with minimal overhead and helps to address scalability concerns as the routing and update are equally demanding. The only bottleneck arises from the locking of edges to ensure a consistent state and eliminate data races, as discussed in Section 4.2.4. However, by having the same worker do both routing and updating sequentially, we minimize the bottleneck edge locks introduced by making sure the edges get unlocked as quickly as possible.

The first task of the routingWorkers is to find a route across the network for the requested chunk. Since the routing process mainly involves reading static values from the global state or the request object grabbed from the channel shared with the requestWorker, the routingWorkers routing logic can be executed concurrently with minimal difficulty and overhead. This leads to significant improvements in execution time compared to our initial adjusted designs.

After a routingWorker has finished routing, it subsequently executes the appropriate update logic based on the output from the routing. This involves updating the balances of the edges in the graph, updating the result counters for both successful and different failures, as well as the logic for the optional waiting, retry, and caching. All of which can be done concurrently with the other routingWorkers due to the changes made in Section 4.2.4. Since the responsibility of updating the timestep and originorIndex was moved to the requestWorker, that also means one less point of contention.

Although most of the routing logic remains unchanged, our new design allowed us to make several improvements. For example, we added the ability to forgive edge debt during the routing process by relocating the forgiveness logic from the update phase. We chose to perform the forgiveness logic when verifying the edge thresholds to better align with how Swarm [24] handles threshold failures, where forgiveness is done by request when close to the threshold. This was made possible by our decoupled design, which separates the Policy and Update layers from cadCAD. Section 5.5 further delves into how we implemented this and the reasoning behind it.

In conclusion, the routingWorkers play a crucial role in executing the simula-

tion logic effectively. By allocating the majority of enabled workers to be routingWorkers, we optimize the performance potential of the simulation. The inherent concurrency nature of routing allows for efficient parallel execution, along with the enhancements made to ensure thread safety in the update logic. This enables the routingWorkers to operate concurrently with minimal overhead, primarily limited by the hardware capacity and the necessary edge locks for synchronization. The design choice of maximizing the utilization of routingWorkers contributes to the overall efficiency and scalability of the simulation.

4.4 Introducing Epochs

One of the challenges of utilizing concurrency is the inherent randomness associated with using goroutines. Since we cannot predict in which order they will get time on the CPU, we cannot determine the order in which they will complete their tasks [8]. In our simulation, the timestep counter is continuously incremented and assigned to each created request object in order. To maintain consistency with PySwarmSim, we have kept the assumption that 12500 timesteps are equivalent to one second. This allows for the idea that routings for requests that occur within each interval can overlap each other and effectively happen at the same time.

However, during the transition from one second to the next, we want to ensure that requests from different intervals are not fulfilled out of order. If newer requests get routed before older ones due to the unpredictability of the goroutine completion order, it can lead to inconsistencies, inaccuracies, or other issues in the simulation.

To address this issue, epochs, and the accompanying logic was introduced and incorporated into our design. An epoch is a separate counter that tracks the number of seconds passed, with each epoch being equivalent to 12500 timesteps. The use of epochs provides a clearer and more expressive way to switch to a new second rather than relying on the modulus operator and comparing *timestep%12500*.

When a new epoch begins, we added a temporary pause in the program to allow the routingWorkers to finish their current requests from the previous epoch. Then only when all routingWorkers are finished will the requestWorker start up again and produce new requests in the new epoch, which will be subsequently picked up by the routingWorkers again. This synchronization step ensures that

all requests generated in the current epoch are completed before moving on to the next, thereby maintaining the correct number of iterations in each interval.

By ensuring that all routings are synchronized at the end of each epoch, the simulation can minimize inaccuracies that may arise due to concurrent routings completing out of order with respect to their assigned timestep. We observed additional advantages by maintaining consistent usage of epochs throughout the design, and we will elaborate on them in subsequent sections.

4.5 Extract Data

During the simulation, a significant amount of output is produced, consisting mostly of numerical data that is relevant to studying bandwidth incentives in Swarm [24]. In order to preserve this information, it is necessary to extract and analyze the data. This section will explore how GoSwarmSim extracts the different data generated when running the simulation.

4.5.1 StateFlushWorker

Throughout the simulation, the global state undergoes slight modifications following each iteration. Although the ultimate state is printed upon completing all iterations, retaining the state's evolution during the simulation run may be desirable.

The stateFlushWorker is its own separate goroutine worker. During the RoutingWorker execution of the update functions, values associated with the new state are returned. For the stateFlushWorker, the values of interest include the counters for the current number of retries, the current number of cache hits, the current number of successfully found chunks, and more. These values get bundled up together into a new object along with the timestep and get subsequently sent to the StateFlushWorker over a new channel.

Since there can be a relatively large overhead of continuously writing data to disk, the stateFlushWorker only prints the values associated with the current state every **N** iteration. After finishing a simulation run, these values can be plotted against the timesteps to see the evolution over time. One can decide the interval themselves, to increase the accuracy of the plot.

An example of using these values is shown in Figure 4.6 with each interval

being one epoch. Where we can see behavior expected by caches, where the cache hits start slow, but as the caches increase, the number of hits increases.

4.5.2 RouteFlushWorker

The same approach discussed in the previous section is also used for the routes generated throughout the simulation. While we conduct a significant portion of the necessary analyses and calculations in-process for the routes, it can be highly advantageous to retain all the routes in a file for future usage.

To minimize the possibility of the flushing process becoming a bottleneck compared to the numerous routingWorkers continuously generating request outcomes, the saving of routes is executed using a goroutine worker named routeFlushWorker. The routingWorkers sends the routes to the routeFlushWorker via a channel, and the routes are flushed to disk in JSON format that corresponds to the route struct. The route struct includes the epoch, the chunk, the routing result, and the route.

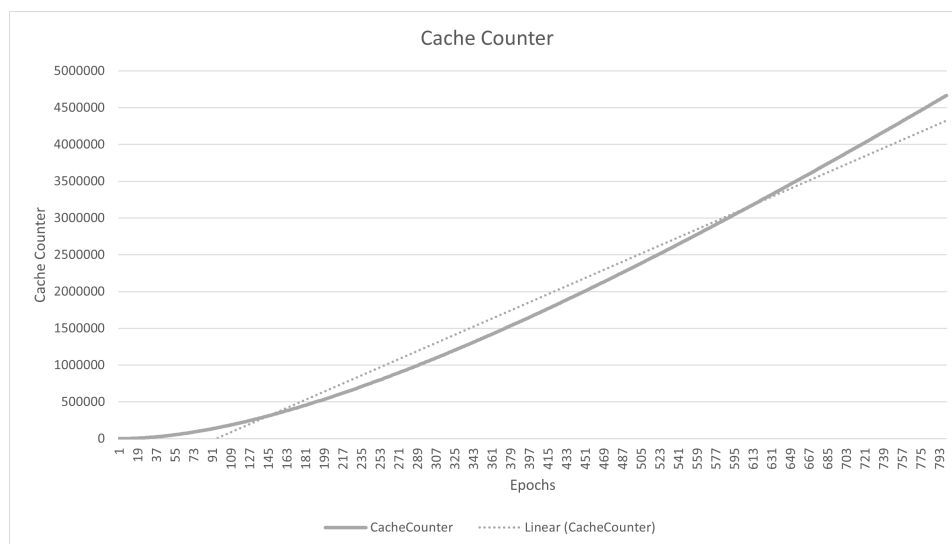


Figure 4.6: Evolution of cache hits during a 10 million simulation run.

4.6 Analyzing Data

PySwarmSim continuously writes its outputs to the terminal, which can then be saved to a file for further analysis. For instance, an output file created with 250,000 iterations has a size of 34.5MB, which is relatively large because of the utilization of string format to store the output. As the performance of the GoSwarmSim improved the issue became more significant. If the number of iterations was increased to *1billion*, following the same linear increase in file sizes, the resulting output file would be roughly 138GB. In this section, we will examine how GoSwarmSim enables the analysis of the generated output.

4.6.1 OutputWorker

To address the issue above, we introduced another worker dedicated to the output. The outputWorker, similarly to the workers in Section 4.5, operates as a separate goroutine and receives data continually through a designated channel from the routingWorkers. The outputWorker provides two different options, enabling users to select the output they desire.

The first option is to write the output that was previously printed to the terminal to a dedicated file using Go's native *bufio* package [8], which provides buffered writing to disk. This option offers faster writing to files compared to redirecting terminal output.

The other option is to perform calculations on the output in memory and only write the results to a file. Various calculations are available, which can be treated as distinct experiments. Each experiment has its own dedicated file, and the resulting outcomes are written to that file. This approach eliminates the problem of large files for a high number of iterations and eliminates the need for intensive post-run calculations on the output.

The in-memory calculation works by saving the output in memory, and when a counter reaches, for instance, 100,000, the calculations are performed on the output, and the results are written to a file. The output that was in memory is then flushed, and the process repeats every 100,000 iteration. This method ensures that only the calculation results are written to the file, making it possible to calculate the average of all the results at the end.

4.7 Final GoSwarmSim Design

The previous sections have discussed the different approaches we explored and the ways in which our approach differs from PySwarmSim. The use of goroutine workers provided us with increased flexibility and opportunities to modify and enhance GoSwarmSim. In this section, we will delve into the impact of our final approach on the simulation steps and the overall design of the simulation.

4.7.1 GoSwarmSim Simulation Steps

As previously discussed in Section 4.3 dedicated to the goroutine workers approach, changes were made to the simulation steps in GoSwarmSim. Figure 4.7 illustrates the simulation steps of GoSwarmSim, with three specific parts highlighted. These parts correspond to the changes in the GoSwarmSim simulation steps, which differ from those of the PySwarmSim simulation steps discussed in section 3.2 and illustrated in Figure 3.1.

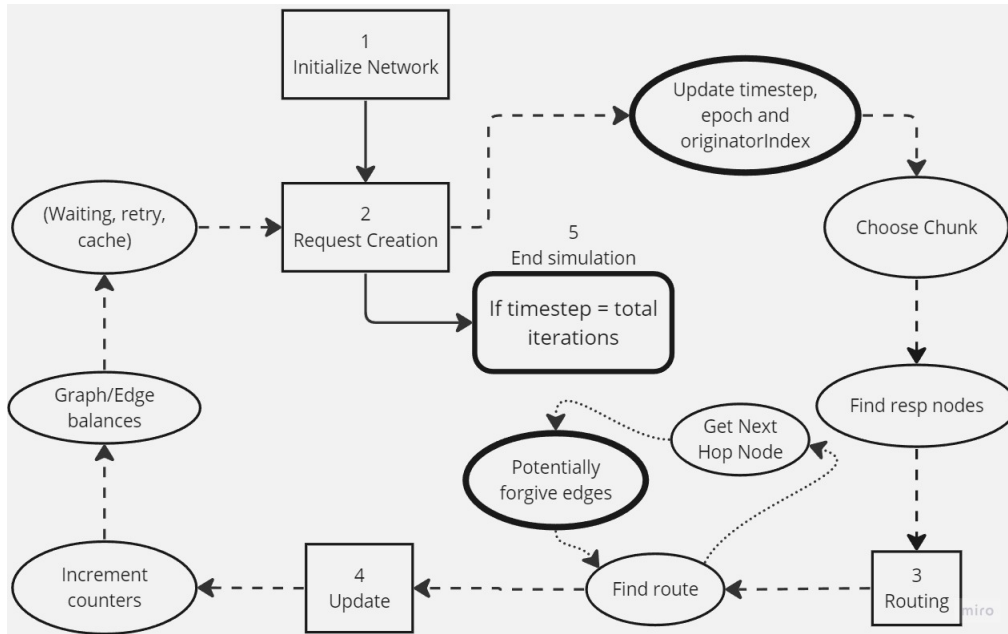


Figure 4.7: GoSwarmSim simulation steps

4.7.2 GoSwarmSim Design

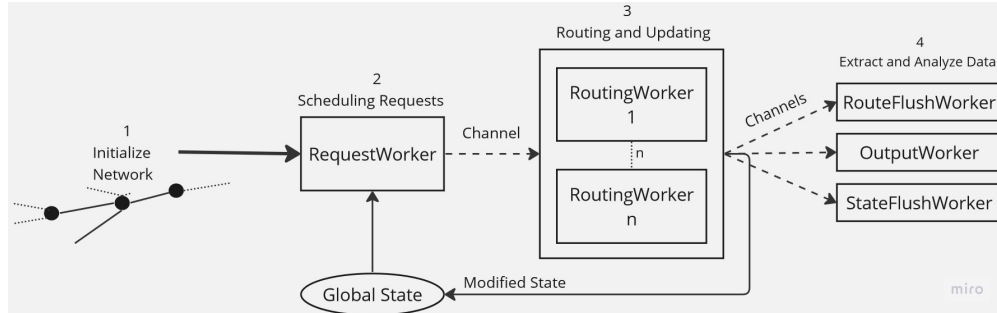


Figure 4.8: GoSwarmSim design

The final design of GoSwarmSim is illustrated in Figure 4.8. GoSwarmSim consists of four general parts. The first part in Figure 4.8 is Initializing the Network using a created file that outlines the various specifications for the network. It continues with Scheduling Requests in part 2. Part 3 starts with the request-Worker concurrently and continuously creating requests that are sent over a channel to the routingWorkers. In turn, the n routingWorkers continuously grab requests from the channel, perform the routing for the chunk in the request, and execute the corresponding update logic. After the RoutingWorker has finished updating, it has the option to send results based on the routing outcomes, the current global state, or other data over multiple channels to the stateFlushWorker, routeFlushWorker, and outputWorker. Part 4 consists of the mentioned three workers, each responsible for different aspects of the simulation data.

4.8 Conclusion on Adjusted design

The decision to replace the tightly coupled logic derived from cadCad with a decoupled main loop proved to be critical in improving our design. The decoupled logic allowed us to explore and discuss several approaches for integrating concurrency into our design. Out of various the approaches considered, incorporating goroutine workers and implementing the producer-consumer pattern proved to be the most optimal. In addition, the use of goroutine workers provided a solution for adjusting other aspects of the design that were focused on extracting and analyzing the simulation data.

Chapter 5

Implementation

During the process of translating the code from Python to Go, adjusting the design, and trying to enhance the simulation in multiple ways, we implemented many changes and enhancements.

This chapter will focus on why we made the changes we did, how we did them, and showcase the benefits of the changes introduced. Specifically, this chapter focuses on the following. Some challenges we faced when directly translating Python logic to Go. How we decided to reference the objects used, for instance regarding the nodes, global state, and graph. Specific changes were made to improve code clarity and readability. Our decision to only search for the next-hop node in the appropriate adjacency list as determined by the XOR distance. Improved the waiting feature by adding a queue of chunks instead of only a single chunk, and increased the waiting time to increase the chance of successfully finding the chunk. Similarly, improved upon the retry feature by altering which nodes are skipped in the retries to improve the chances of successfully finding the chunk. The algorithm used for the calculation of responsible nodes was also substantially improved by incorporating binary search and limiting the subset of nodes that get their XOR distance to the chunk calculated. Lastly, we moved the configuration settings from a constant object in the code to a separate yaml file.

5.1 Challenges when Translating Python Logic

Although the chapter's main topic is about specific implementation enhancements it's worth mentioning how the direct language translating process was done. Due

to the significant difference in the language’s philosophies, it wasn’t always feasible to directly port the Python logic to Go without some modifications.

Python and Go both have fundamental data types and structures, but Python is dynamically typed [11], while Go is statically typed [8]. As a consequence of this difference, we often need to split variables used in Python into many variables or convert them to a Go structure object, which we will refer to as a struct going forward.

PySwarmSim’s use of ordered sets, which come from the custom package **ordered_set** [15], also poses a challenge when translating Python code to Go. The `OrderedSet` is defined as a custom `MutableSet` that remembers its order so that every entry has an index that can be looked up. However, since Go does not have this data structure, we will need to create an alternative solution to emulate the same functionality.

Another challenge is translating Python-specific functions that have no native equivalent in Go. For example, the usage of **if x in y** in Python is actually an interface that allows for looping over a set of variables and checking if an element is present. As a rule, Go does not want to include hidden loops in its native functions [8]. Therefore, we had to create a custom function called **Contains** that implemented the same functionality.

5.2 Referencing Objects

PySwarmSim uses the languages’ classes and objects for the network and the nodes. These objects are commonly passed as arguments to functions, simplifying the process of accessing and modifying their properties. When objects are passed as arguments to functions in Python, the language does not create a copy of the object’s values. Instead, it creates a reference to the original object. In fact, even simple values are treated as references, meaning that all objects, regardless of their complexity, are actually references [26]. Since a reference in Python is similar to a pointer in other languages, it behaves in a similar way, which is why there isn’t a significant performance loss when passing large objects to multiple functions. The following subsections explain how GoSwarmSim overcomes this problem.

5.2.1 Value objects vs pointers in Go

In contrast to Python, Go has a clear distinction between values objects and pointers to those objects. When a node structure object (struct) is passed to a function, the values in the fields are copied rather than referenced [8]. Just as making copies can have drawbacks and benefits, using pointers in Go can have both positive and negative consequences. The distinction between value objects and pointers to objects allows for context-based optimizations. The network, or graph in our case, which is a sizeable struct, can be passed as a pointer, while the smaller struct for requests can be passed as a value. Having the pointer to the graph, we can both read from and modify the fields of the original global object, as when updating the edge balances. However, when dealing with request structs that are only read from and never modified, it makes sense to pass them as value objects. This is because modifying a value object doesn't affect the original.

5.2.2 Drawbacks of pointers

While using pointers in Go can reduce the memory usage when dealing with large objects, it can also increase the risk of errors and make debugging more difficult [17]. For example, invalid memory addresses or nil pointer deference can cause issues and tend to be relatively hard to diagnose and fix. Alternatively, the copying of values can be less than the overhead of creating a reference and is optimal for small objects that have more reads than writes. However, as we are often dealing with distinct values for reading, passing them as such is less prone to errors, and in case of an error, it is easier to diagnose and identify the root cause.

Due to these challenges, we decided to reference nodes and chunks using their id or address in the form of an integer value type. We continued to reference the graph, the global state, and the node objects themselves with pointers. The global state includes a reference to the nodes map used for initializing the graph, where pointers to the node objects are stored with their id value serving as the key. This allows us to easily retrieve the node objects and access their respective fields whenever needed, such as during the routing process when accessing a node's adjacency list.

5.3 Improving code clarity and readability

PySwarmSim was developed as a prototype, prioritizing functionality over code clarity, readability, and performance. As a result, some aspects of the tool were implemented in a suboptimal manner, with solutions being chosen based on the first attempts that worked rather than through thorough evaluation and optimization. In our simulation, we made a deliberate decision to prefer using structs (structured objects), instead of using similar data structures as Python, when dealing with objects containing multiple value types. This decision was made to improve code clarity and maintainability, as well as to reduce the complexity of handling large nested logic. By separating the various values of an object into a struct object with named fields designed specifically for those values, it becomes easier to understand the purpose and meaning of each field, and the code becomes less prone to bugs and errors. An example of this is the change made to how the route data is stored and used. Although this change has no effect on the tool's results, it greatly improves the code's maintainability and readability which is presented further in this section.

5.3.1 Objects of variable lengths containing multiple types

In PySwarmSim, the route object that is created during routing, described in Section 3.2.3 consists of an originator, zero or more next-hop nodes, and the chunk. However, in addition to these regular values, the route may also include the special values **-1**, **-2** and **C**, which serve as aliases for the two types of failed requests and for routes where the chunk was found by utilizing the cache.

These aliases are utilized to determine the specific logic that needs to be executed during the update phase. As you delve deeper into how the logic uses these values, it becomes apparent that the implementation is unnecessarily complex and can become difficult to comprehend in this regard. Working with a list of varying lengths that contain values of different value types, each with its unique significance requires careful consideration of both normal and edge cases to accurately extract and use the intended values for their intended purpose.

5.3.2 Route list to requestResult struct

Considering the route example in Section 5.3.1, the route list was refactored into a **requestResult** struct [8]. This struct includes fields for the route, which is now represented as a simple list of addresses for the nodes with the originator always being the first in the list and a separate field for the address of the chunk the originator requests. The struct also unifies other values that were previously returned as distinct variables, as seen in the following code segment.

```
type RequestResult struct {
    Route          []NodeId
    PaymentList    []Payment
    ChunkId        ChunkId
    Found          bool
    AccessFailed   bool
    ThresholdFailed bool
    FoundByCaching bool
}
```

By having the values as distinct fields within the struct in GoSwarmSim, we can easily access and modify the values we need. This makes the code more readable and understandable for other developers and users, as the meanings of the values are clearly defined and separated from one another.

When the requestResult struct with the route reaches the update logic for updating the edges, we see the difference it makes to the readability of the code. As shown in the code segment for PySwarmSim, the route is checked if **-1**, **-2**, and **C** exist to ensure they are not counted as nodes in the following logic, and then have different logic paths for the different cases. Contrary to what we see in the code segment for GoSwarmSim where we only need one logical path to execute the same functionality. This makes the code more readable and understandable for other developers and users, as the meanings of the values are clearly defined and separated from one another.

```

PySwarmSim:
    if -1 not in route and -2 not in route:
        if 'C' in route:
            chunk_id = route[-2]
            # The last one is chunk id, second last is storage node
            for i in range(len(route) - 3):
                requestor_node = route[i]
                provider_node = route[i+1]
                ...
        else:
            chunk_id = route[-1]
            # The last one is chunk id, second last is storage node
            for i in range(len(route) - 2):
                requestor_node = route[i]
                provider_node = route[i+1]
                ...

GoSwarmSim:
    if requestResult.Found {
        for i := 0; i < len(route)-1; i++ {
            requesterNode := route[i]
            providerNode := route[i+1]
            ...
        }
    }

```

5.4 Finding Next-hop Node

In PySwarmSim, when finding the next hop node to add to the route, the simulation searches through all of the originator's adjacency lists. All the nodes kept within are compared to each other with the goal being to find the node with the least distance to the chunk. However, we discovered that this approach involved looping over a larger dataset compared to searching within only the appropriate adjacency list determined by the XOR operation.

With that in mind, we decided to adopt the same approach as Swarm [24] and

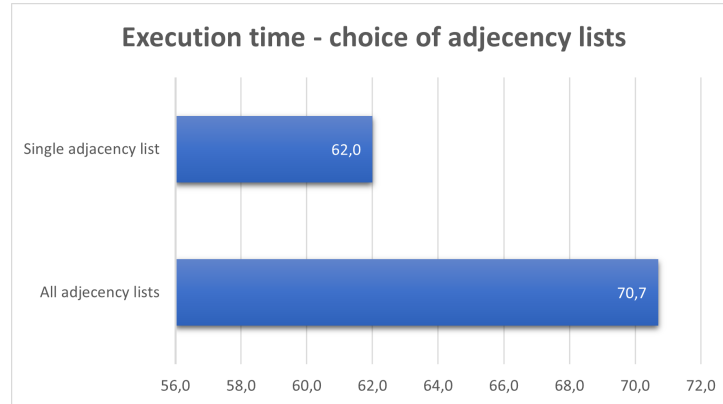


Figure 5.1: Showing the average execution time searching all adjacency lists compared to only the appropriate one, from 20 runs each with 10,000,000 iterations

only consider the nodes within the appropriate adjacency list. After conducting tests, we determined that the outcomes of the two versions were mostly identical, if not better when only searching for the appropriate one. Additionally, our approach was faster as seen in Figure 5.1, and resulted in less wasted computational power. This observation indicates that Swarm’s approach, which relies on a routing table of adjacency lists where the appropriate adjacency list is determined by the XOR operation consistently leads to a closer node and is indeed the most optimal approach [19]. As a result, we believe that our approach, which aims to better emulate Swarm’s behavior, is a superior version.

5.5 Forgiveness of Edge Debt

In Swarm [24], if adding the cost of a request would cause a node to exceed the edge debt threshold, the requester will also send a forgiveness request. After the responder receives the forgiveness request, the recipient node evaluates it and decides whether or not to forgive some of the debt on their shared balance. If the recipient node decides to forgive some of the debt, it proceeds to fulfill the request from the requesting node. In PySwarmSim, this is done a different way. Due to the nature of cadCAD [12], which involves two separate and disjoint parts for reading and updating the state, the simulation did not forgive the nodes during routing. Instead, it had to keep track of which edges hit the threshold limit during the routing process and include it in the result that the update part uses as input.

Then in the update step, go through all the saved edges and evaluates whether or not they fulfill the following criteria for partial forgiveness of their edge debt.

$$\frac{CurrentTimeStep - LastForgivenTimeStep}{RequestPerSecond} > 0 \quad (5.1)$$

- *RequestPerSecond*: as the constant with a value of 12500.
- *CurrentTimeStep*: as the variable for the current iteration.
- *LastForgivenTimeStep*: as a variable stored on the edges, and keep track of the timestep the edge was forgiven last.

Although the method is functional, it has some limitations regarding the frequency of routing attempts for chunks resulting in threshold failures due to the imbalance of directed edge pairs debts. For instance, if the maximum threshold limit is set to 16, then the difference in edge debt between the opposite edges of the same nodes cannot exceed that value. Moreover, the forgiveness criteria for an edge are evaluated only after first checking the threshold. This involves verifying whether adding the cost of fulfilling the request to the edge debt would cause it to exceed the threshold limit. Consequently, an edge approaching the threshold can only be considered for forgiveness if it first fails the threshold check and is subsequently sent to the update phase. This approach leads to a higher frequency of requests resulting in threshold failures than is realistic, thereby undermining the purpose of a simulation. How GoSwarmSim improves the forgiveness of edge debt is explained further in the upcoming sections.

5.5.1 Forgiving during routing

To better reflect the behavior of Swarm [24], we made the decision to evaluate the forgiveness criteria and execute the forgiveness logic of the debt during the routing process. This will allow us to determine whether the edge debt balances exceed the threshold limit, and if so, potentially forgive parts of the debt, all in one step.

The new approach follows the following logic: First, it verifies whether the edge debt exceeds the threshold limit, similar to PySwarmSim. If the threshold check fails, we proceed to evaluate the forgiveness criteria. If the criteria are met and the debt is partially forgiven, we retry the threshold check with a higher

chance it will succeed this time. However, if the debt cannot be forgiven, the outcome will still be a threshold failure. The possible outcomes of the new logic include a normal successful routing attempt, a normal threshold failure, a successful routing attempt with partially forgiven debt, or a threshold failure with reduced debt. Unlike PySwarmSim, three out of the four outcomes are valuable and will improve upon future communication between the two nodes on the edge.

By making these changes, we have eliminated the requirement for a request needing to fail the threshold check before it can succeed in a future attempt, due to the forgiven debt. Evaluating the forgiveness criteria during the routing process, we can identify and forgive the debt that exceeds the threshold limit at the same time, thereby enabling successful routing attempts without the need for a prior failure. The outcome is a reduction in the number of requests that result in a threshold failure, which increases the likelihood of more successful routing attempts. Furthermore, we are of the opinion that our version of the forgiveness logic better reflects how amortization works in Swarm, making the simulation more accurate.

5.5.2 Frequency of checking threshold

Another improvement we made, thanks to the forgiveness logic changes, is limiting how often the edges need to get their threshold checked during routing. Since PySwarmSim relied on edges failing the threshold check and subsequently getting sent to the update part for forgiveness, it made sense for them to conduct the threshold check on more edges than necessary. Since we do not have that constraint, we only need to check the threshold for an edge if it is a genuine candidate for the next hop node.

In practice this means our logic is as follows: When finding the next hop to add to the route, we loop over all nodes in the current last hop node's appropriate adjacency list. We ensure that the distance to the chunk is closer than the current last hop node for each node, and skip them otherwise. Similarly, we compare the distance of the following nodes to the contender and skip them if the current contender is closer to the chunk. This is the main difference with the logic from PySwarmSim, as they would perform the threshold check before verifying if the node is closer than the current contender. This change, together with the changes explained in Section 5.4, improves upon the execution time needed to find the

next hop to add to the route, therefore improving performance.

5.5.3 Epoch in forgiveness

The forgiveness criteria were also changed from PySwarmSim as seen in Equation 5.1, by incorporating our new epoch explained in Section 4.4. Rather than using intervals with a minimum of 12, 500 iterations as the criteria, we assumed that every edge should have an opportunity for forgiveness every second, or each new epoch. That means instead of keeping track of when an edge was last forgiven, in terms of the timestep or iteration number, we keep track of the previous epoch instead. Then during routing, we only check if the current epoch is later than the previous epoch, as seen in Equation 5.2. As a result, we have effectively decoupled the forgiveness logic from the requests per second variable, allowing us to handle forgiveness and the number of requests per second independently.

$$CurrentEpoch - LastForgivenEpoch > 0 \quad (5.2)$$

- *CurrentEpoch*: the current epoch, brought within the request object.
- *LastForgivenEpoch*: the epoch when an edge was last forgiven, which is stored in the edge attributes.

This approach gives the edges a higher chance to get forgiven twice during a subset of timesteps, reducing the consequences of edges being popular for short periods of time. For example, if an edge were close to failing a threshold check both at the end of an epoch and the start of the next epoch, it now has the chance to get forgiven both times. Doing it this way increases the flexibility of the forgiveness, while at the same time not increasing how many times an edge can be forgiven in total during the entire simulation run. Comparing our approach to that of Swarm [24], we observe that the modifications we made to the forgiveness and amortization process bring our version closer to reality, surpassing the PySwarmSim in its accuracy.

5.6 Cache

PySwarmSim used a global cache set, which uses the OrderedSet mentioned in Section 5.1. This is a sensible method for simulating the caching we see in Swarm

[24]. It is possible to access all the cache information in a single variable, making it easy to reference in the code or retrieve the information after the simulation has concluded for further analysis. Directly applying this logic to Go using a map and our added concurrency produces **concurrent map read and write** issues because the data structure is not meant to be accessed by multiple goroutines simultaneously [8]. However, introducing a mutex lock to the globally shared cache map to resolve these issues can lead to significant overhead and a much higher level of complexity to ensure that the interactions with the object are thread-safe [8]. How we overcame these problems is explained further in this section.

5.6.1 Atomic maps

In Go's standard library sync package [13], there are atomic maps available. These maps, like other atomic functions, are designed with concurrency in mind and facilitate normal usage while keeping the operations thread-safe behind the scenes. Two goroutines can simultaneously read and modify the same map item without issue, as the atomic map would queue up the operations and execute them in an appropriate sequential order. However, during our testing, we found that the performance hit was worse than expected.

The atomic maps employ a strategy where write operations are only performed after queued-up read operations have finished. According to Go's documentation [13], atomic maps are intended for use cases where a key's values are only written to once and read many times, or where multiple goroutines have their own set of keys.

5.6.2 Local objects on nodes

With this in mind, we decided to store the cache data on the nodes themselves. One of our motivations was again to try to make the simulation closer to Swarm [24], in this case where the nodes function as separate entities responsible for maintaining their own cache and using it for cache hits. By storing the cache on the nodes, if a goroutine needed to access a node's cache, it would only need to refer to the objects fields of the node itself, removing the overhead created when fighting for the global mutex lock. There is however a possibility of two goroutines attempting to access the same cache map from the same node simultaneously. Although our tests showed this being unlikely, we decided to include a mutex

lock for each cache map on the nodes to ensure that there would be no data race issues, at the cost of a slight increase in memory usage.

By having the nodes only reference themselves while checking cached chunks, and also when adding new data to the cache, the concurrent map read and write is no longer an issue. Additionally, by removing the need to lock the global cache map, we were able to eliminate the bottleneck and the performance hit with cache enabled was negligible.

Lastly, since we don't have access to the caches directly from the global state anymore, we added a field to the state that tracks the number of cache hits during the simulation. We implemented this using atomic functions [13] for incrementing and loading the values, similar to the other counters. This approach ensures low overhead when writing to or reading from the variable using multiple goroutines simultaneously, while also maintaining thread safety.

5.7 Waiting and Retry

If a request for a chunk fails, it could be due to one of two possible scenarios. One is an access failure, which occurs when the chunk can not be found during the routing. The other is a threshold failure, which happens when the edge debt threshold limit is reached between the requestor and the responder peer during the routing for a chunk.

To counteract the possibility that these two failures may occur, we have implemented and improved upon the waiting and retry features of PySwarmSim. The simulation configurations allow the user to enable or disable them at will by setting the **waitingEnabled** and **retryWithAnotherPeer** options to true or false. The two features can be utilized either independently or in combination, but it is important to note that the simulation's outcome and results will differ depending on which approach is taken.

As our implementations of GoSwarmSim diverged further from PySwarmSim, we realized that the waiting and retry logic needed to be adjusted to align with the other changes. This was particularly important due to the concurrent nature of the requestWorker and routingWorkers, which required us to prioritize thread-safe code. However, the most significant changes we made were related to what we perceived as major design flaws in PySwarmSim. We felt that the intended behavior of the waiting and retry logic was not being accurately reflected in the

code and therefore subsequent simulation results, and therefore made significant changes to address these issues.

This section will first detail the changes to waiting, where instead of keeping a single chunk for each originator, we store multiple chunks and only retry them after the chance of succeeding is substantially higher. Next, we will detail the changes to retry, where we chose to reject and skip the last-hop nodes in the route rather than the first-hop nodes. Lastly, we will discuss the results shown when combining the

Similarly, improved upon the retry feature by altering which nodes are skipped in the retries to improve the chances of successfully finding the chunk.

5.7.1 Pending queue of chunks

In PySwarmSim, waiting is implemented by using a dictionary [11] called **pending_dict** with a key-value pair consisting of the originator and a single pending chunk. The pair gets added to the global dictionary after a routing attempt results in a threshold failure as a result of the edge debt exceeding the threshold limit. When it's time to initiate a new request from the same originator, the corresponding pending chunk in the dictionary will always take priority over selecting a new chunk. Consequently, in every subsequent iteration with the same originator, the routing process will focus on the same chunk until it is found, after which the originator and the chunk will be removed from the global pending dictionary. This is a simple and relatively effective way to implement the waiting logic, but it also comes with quite a few drawbacks because of its simplicity.

The waiting time before rescheduling the same chunk as before in PySwarmSim is relatively short. As the time spent waiting only corresponds to the time it takes for the simulation to have looped through all the designated originators, ending up back at the initial originator. Given that the last edge in the initial routing attempt for the chunk resulted in a threshold failure, it is likely that any subsequent routing attempts by the same originator for the same chunk would create the same result. As detailed in Section 5.5, the reason for the edges failing the threshold checks is the imbalance of the edge debts, which will naturally amortize over time. Alternatively, the edge can wait for the opposite edge to accumulate enough debt to restore the balance. However, in both cases, it may take some time for balance to be achieved.

With this in mind, we decided to extend the waiting period for pending chunks before initiating new routing attempts for them. Given the incorporation of epochs into the forgiveness of edge debts, it also makes sense to keep the chunks waiting until the start of a new epoch, which consequently increases the chance for new routing attempts to succeed. Moreover, this approach provides the originator with the flexibility to continue requesting new chunks during the waiting period, as routing for new random chunks generally has a higher chance of success than retrying previously failed ones. As we now require waiting for a new epoch before selecting a pending chunk, we must keep track of all the threshold failed chunks requested by an originator during an epoch, rather than just a single chunk. Furthermore, due to the requirement of waiting for a new epoch before selecting a pending chunk, it is necessary to keep track of all the threshold failed chunks requested by an originator during an epoch, instead of just a single chunk.

Similar to the changes made to cache detailed in Section 5.6, we decided to move the global pending dictionary logic to the nodes themselves. To meet our requirement of storing multiple chunks per node, we have implemented a struct object for each node that includes a pending queue and a variable to keep track of the index of the last searched chunk in the queue. Every item in the pending queue is a queued chunk struct which consists of the chunk address itself, a counter for how many times this chunk has been searched, and a value for the last epoch it was searched for. When the requests are created for the routing with waiting activated and the originator's pending queue is not empty, it will pick a chunk from the queue instead of choosing a new one. However, queued chunks are only valid to be searched again if the current epoch is later than the epoch value stored in the queued chunk struct. In other words, each chunk stored in an originator's pending queue is given an additional routing attempt at the beginning of a new epoch, thereby maximizing the chance of successful routing for each subsequent attempt.

5.7.2 Retry with alternative routes

In PySwarmSim, similar to both waiting and cache, retry is implemented by using a dictionary [11] called `reroute_dict`. The keys in the dictionary are the originators, while the value object is a collection of first hop nodes from the originator, used in previous routing attempts that failed to find the chunk. The goal of this

feature is to ensure that with each new routing attempt for the same chunk, a different first hop node is selected, with the intention of discovering a new route that can lead to the chunk. The chunk related to the reroute takes precedence over both waiting chunks and new chunks, so it continues being picked for routing until it's found or until one has exhausted all the options for first hop nodes in the originator's adjacency list. Similar to waiting, this implementation also has some limitations and drawbacks which we wanted to address and improve upon.

Following the decision to have the nodes themselves keep track of the cache and the pending queue, we also moved the reroute logic from a global object to local ones kept on the nodes. This again made the most sense to minimize the need to constantly lock and unlock the global object for each read and write operation. Although we are sacrificing the option to effortlessly retrieve all the reroutes from a single global object, we believe it is a worthwhile trade-off given the advantages of improved performance and localized data access.

When moving the logic to the nodes, we also made the following changes. Each originator has their own reroute struct that consists of a current reroute with the chunk address and the rejected nodes associated with the previous unsuccessful routing attempts for that particular chunk. While that is similar to PySwarmSim, we also keep a history of the chunks and their corresponding rejected nodes as a map [8]. If a failed chunk has been retried several times without success, the routing for the chunk is deemed a true failure, and the originator next time proceeds with a new chunk. In the case a new chunk has previously been deemed a true failure, the reroute will utilize the chunk's history and the corresponding list of rejected nodes for future attempts, ensuring that the routes taken this time are different from the previous ones. Similarly, if the chunk is found after rejecting a number of nodes during the retries, all future routings for the same chunk address will have the benefit of bypassing the rejected nodes stored in the reroute history of the chunk. That means the routings for the chunk will succeed on the first attempt without the need to redundantly re-attempt all the failed routes.

Another enhancement we made to the retry logic is changing which nodes from the previous routes get placed in the rejected nodes list. While PySwarmSim rejected the first hop node, we decided to instead skip to the last hop nodes. Specifically, the nodes added to the rejected list are those responsible for the failures in the previous routing attempts. The change was made in response to a potential issue in PySwarmSim where even if the first hop node was skipped in

the previous route that failed, there is still a chance to reach the same last hop node and experience the same failure again. By consistently removing the last hop nodes that caused the previous failures, we can guarantee that each subsequent rerouting attempt has a fresh chance to locate the chunk without repeating the same mistake. The suspicion was validated by examining the retried routes while rejecting the first hop nodes, which revealed that the same failing last hop nodes continued to appear. This approach increases the probability of success and reduces the likelihood of encountering the same failure again, ultimately improving the effectiveness and reliability of the rerouting process, as seen in Figure 5.2.

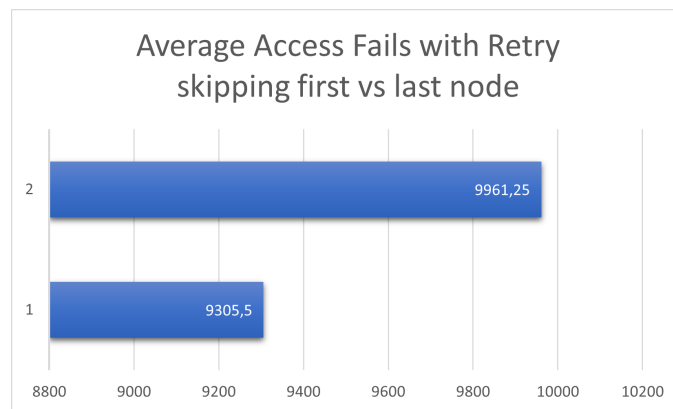


Figure 5.2: Showing the average number of access fails from 20 runs each with 10,000,000 iterations

When it comes to retrying failed routing attempts for chunks, there is always the consideration of not wasting the bandwidth that could be otherwise used for other chunks more easily found. Typically, a chunk can be located after a few retries, but this isn't always the case. As mentioned earlier, we stop retrying for a chunk once we determine that the likelihood of success is low. However, making that decision isn't always easy. While our primary objective is to locate every chunk, we must balance this goal with the need to avoid wasting bandwidth and iterations on chunks that are unlikely to be found.

5.7.3 Waiting and retry together

When running GoSwarmSim with both waiting and retry enabled, taking into account our improvements to both their functionalities, they play to each other's strengths and we get the benefits of both. In the event that a routing attempt for a chunk fails, the chunk is added to both the reroute struct and the pending queue. Retry attempts are made immediately, and if the chunk is successfully routed, it is removed from both objects. However, if the retry is unsuccessful after some attempts and reset, the chunk remains in the pending queue and is retried when the next epoch starts. This cycle continues with multiple retries until the chunk is either successfully routed or deemed impossible to be found.

Considering the point of both waiting and retrying was wanting to find all requested chunks, we concluded that a new configuration option was necessary to accurately reflect if this was achieved. The configuration option we made to achieve this was coined **iterationMeansUniqueChunk**. If routing attempts for the same chunk fail multiple times because of waiting or retrying, each failed attempt is still counted towards the total number of iterations, as well as the number of threshold failures and access failures. This can potentially skew the final results. When running the simulation with this option enabled, the request worker is modified to only increment the iteration counter when a new chunk is randomly chosen for a request, rather than when a chunk is chosen due to waiting or retrying.

The result is an increase in the total number of iterations, but the percentage of access failures and threshold failures relative to the total number of iterations now is more in line with what we would expect. Most importantly, however, the result proves that by running the simulation with both waiting and retry enabled, we can confidently conclude that we are able to locate almost all unique chunks in the end.

While implementing the waiting logic into GoSwarmSim, we found a mistake in how PySwarmSim updated its pending dictionary. We found that the function didn't differentiate between the types of failures that occurred, and consequently added the chunk address to the current originators pending queue for both access failures and threshold failures. This behavior is desirable only when using retry along with waiting. Without the possibility of rerouting, there is no reason for waiting and routing across the network with the same route again in the event of

an access failure.

For this reason, we separated the logic into two sections based on if retry is enabled or not. If retry is enabled, we add a chunk to the back of the originator's pending queue in the case of a threshold failure and an access failure and only remove the chunk if it's successfully found. If retry is disabled, we add the chunk to the back of the pending queue only in the case of a threshold failure and remove the chunk if it's either successfully found or ends up as an access failure. For both cases, we also use the queued chunk counter to remove the queued chunk if it has been unsuccessfully routed more than **X** number of times.

5.8 Calculation of Responsible Nodes

In the following section, we will describe the original method used by PySwarmSim for computing the nodes responsible for carrying the chunk, along with its limitations. Subsequently, we will present our alternative approaches that use a binary search and limit the number of XOR calculations required to find the four closest nodes. Then detail our final approach which overcome the limitations of the original approach and improves performance.

5.8.1 Redundant ad-hoc calculations

In PySwarmSim, the nodes responsible for carrying the chunk in the request are calculated ad-hoc, during the creation of the request. Each time a chunk is selected, it is necessary to iterate through all nodes in the network to calculate the distance between them and the chunk. After which all the distances are sorted into a heap using the **heapify** function of the **heapq** Python package [21]. The heap maintains the distances sorted, enabling $O(1)$ retrieval of the four nodes with the least distance after every distance has been calculated. Unfortunately, this still leads to two $O(n)$ time complexity operations for each request that involves a randomly chosen chunk, due to the initial XOR distance calculations and subsequent sorting.

The major issue with this approach is however when the calculation has to be repeated, even if a chunk address has been through this process before. This results in the simulation unnecessarily performing the same expensive calculations multiple times, leading to a waste of computational resources. With the number

of iterations the simulation is set up to complete being close to the address range size and therefore the number of possible chunk addresses, the cost of picking the same chunk-id twice is relatively low. For example, with the iterations at 100,000 and the address range at 2^{16} which equals 65,536, each chunk will be picked on average 1,524 times. However, as the number of iterations grows, the redundant computation of the closest nodes starts affecting the execution by a significant degree.

5.8.2 Precomputation and improving algorithm

Since this is only a simulation, the network's nodes and the graph's edges remain constant throughout all the iterations, with the edge attributes being the only network-specific values meant to change over time. Because of this, we decided to precompute all the responsible nodes for every possible chunk address in the network initialization phase instead of during the routing. Having a sorted list of all nodes in the network, we utilize a binary search to find the node closest to the chunk address in $O(\log n)$ time [23].

Since we know the list is sorted, we also know the other three closest nodes must be near the closest node returned from the binary search. Therefore we pick an interval of the sorted node addresses with a set number of nodes to either side of the closest node to the chunk in the context of the address space. Then we calculate the distance from every node in the interval relative to the chunk and find the four nodes with the least distance.

After finding the responsible nodes for each possible chunk address, we store them in a list where the chunk addresses serve as the index and the four responsible nodes as the corresponding value. This allows for efficient lookup during the simulation. As a result, when the chunk addresses are randomly chosen during the routing process, finding the responsible nodes for those chunks can be done quickly through a simple index lookup, rather than repeating the expensive distance calculations and sorting.

5.8.3 Interval size returned from binary search

While this approach reduces the execution time required for calculating the responsible nodes, its effectiveness and execution time depends on the chosen size of the interval returned from the binary search. With a network size of 16 bits,

65, 536 possible addresses, and 10, 000 nodes, doing the precalculation with an interval that encompasses all nodes took 20 seconds to complete, taking 200x time to complete compared to 0,1 seconds when using an interval size of 4. Considering the result from calculating the distance between every node and the chunk, we can understand why the original Python version's performance was lacking.

However, a list of sorted distances determined by the XOR calculation on the binary representation of the node addresses does not directly correspond to the same order of the nodes in their decimal format. This means we must be wary of the interval size returned from the binary search needs to be large enough to ensure that the four nodes we identify as the closest to the chunk are indeed the closest to ensure the validity of our results. If the interval is too small, there is a chance that the four nodes we select may not be the actual closest nodes, resulting in a longer routing process and potentially affecting the overall performance of the simulation.

As an example, let's consider an address space using 4 bits, resulting in 16 possible addresses. The first bit determines whether the value falls between the range of 0 to 7 or 8 to 15. As seen in Figure 5.3, for an address in the middle of the address space, like 8, the three closest values to it, according to XOR calculations, would be 9, 10, and 11, instead of the other nearby values like 7 or 6. This is because of how the XOR operation works by matching the bits starting from the left.

Value	0	1	2	3	4	5	6	7
Binary	0000	0001	0010	0011	0100	0101	0110	0111
XOR	8	9	10	11	12	13	14	15
Value	8	9	10	11	12	13	14	15
Binary	1000	1001	1010	1011	1100	1101	1110	1111
XOR	0	1	2	3	4	5	6	7

Figure 5.3: XOR distance calculated for all values with 4 bits to the value 8.

Keeping that in mind, with the non-linear parallel between the sorted distances of nodes based on their binary representation and their decimal format. Finding the optimal interval for our use case was determined by finding the four closest nodes for every possible chunk by doing the XOR operation between the chunk and all nodes. Then determine the maximum interval from the closest node returned by the binary search to the furthest away nodes in the context of

the sorted nodes.

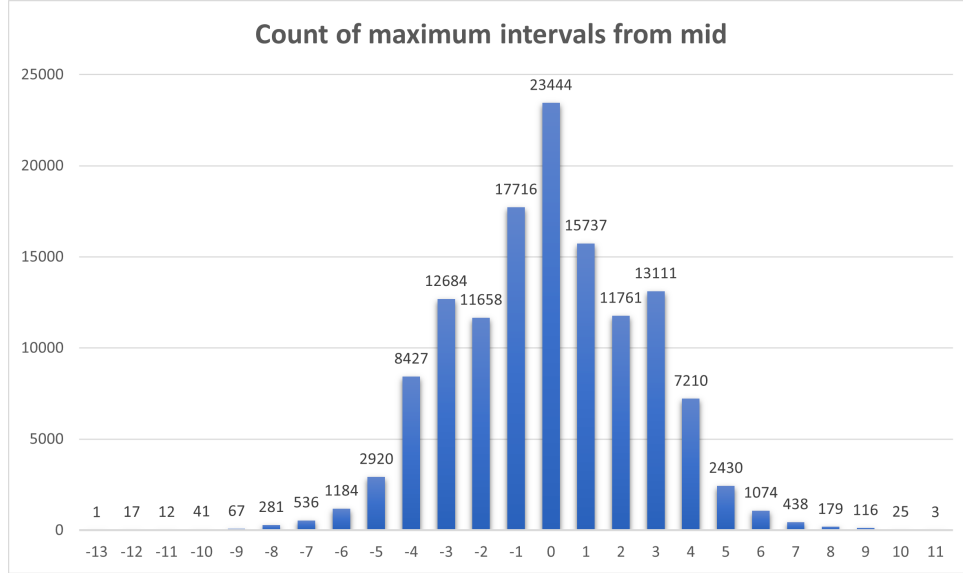


Figure 5.4: The maximum intervals from the closest nodes to the smallest and biggest node in the set of four responsible nodes.

As seen in Figure 5.4, by having a network range using 16 bits, meaning 65,536 possible addresses along with a network size of 10,000 nodes, the biggest interval we saw was 13 nodes from the closest node to the smallest node. Additionally, the farthest interval from the closest node to the biggest of the four closest nodes was 11. This means in this scenario, by having an interval size of 13 you guarantee the four responsible nodes the algorithm finds, are indeed the closest ones. With the possibility this might scale with larger network sizes we did the same test with a network range using 20 bits, meaning 1,048,576 possible address and 20,000 nodes, and we saw the maximum interval grow to 15. Taking this into consideration, and allowing for a margin of error, our final result shows that the interval size should be dynamically chosen, with the size being equal to the number of bits in the network.

5.8.4 Final approach

As this version of the simulation tool is designed to be substantially faster and more robust, it can be run with bigger networks and more iterations than PySwarm-

Sim. When the possibility of a chunk address being picked more than once grows, the performance benefit of doing the computation once grows as well.

The idea of precomputing the nodes described above seemed to be the best option for some time. However, as network sizes and possible node addresses and chunk addresses increase beyond the limits of PySwarmSim, the probability of each address being used for a chunk at least once decreases. For instance, in a simulation with an address space range of 32 bits meaning just over 4 billion possible addresses, precomputing the responsible nodes for all addresses is not viable unless the number of iterations is also higher than 4 billion. Moreover, this puts a significant strain on the system's memory usage as it needs to keep the entire list of the chunk addresses and the corresponding responsible nodes in memory throughout the entire simulation run.

Therefore, we concluded that calculating the responsible nodes for chunks that will never be requested leads to unnecessary calculations once again, much like in PySwarmSim. Although we attempted to speed up the process by performing concurrent precomputation and dividing the address space among multiple goroutines, the reduction in execution time required to justify the unnecessary calculations was not evident.

The final approach was made by combining the previous methods, by picking the best parts of each one. We wanted to keep the lookup possibility with the responsible node values on the chunk address key, but not calculate the responsible nodes and add the key-value pair until we know it was needed. For this reason, the array was also moved to a map [8], since we couldn't rely on the indexes being the same as the addresses anymore. For these reasons, our final version starts the same as the PySwarmSim, where nothing is precomputed. Then when a chunk address is chosen, we look for it in the responsible nodes map first. If the chunk address has been searched before, the map will return the responsible nodes. If it hasn't been chosen before, the calculation of responsible nodes is done and the result is saved to the map for future reference. This way, we get the best of all methods and the minimal amount of computation needed to get the same result.

5.9 Experiments Configuration

In order to ensure consistent experiments and an easier way to overview or change the different running configurations, we incorporated the usage of a `config.yaml`

file. This file provides users with the flexibility to select from a range of predefined experiments or create their own custom experiments. In the event that none of the options provided is selected, the simulation will run with its default values. Furthermore, the user also has the ability to modify configuration options that do not impact the experiment itself.

These options provide an alternative way to easily adjust variables such as the number of iterations, the network size, or the number of active goroutines used. The idea of having config.yaml file for different experiments is to easily switch between configurations and settings.

5.10 Conclusion on Implementation Improvements

In conclusion, this chapter has provided a detailed account of the process of translating PySwarmSim's Python code to Go, as well as the adjustments made to improve GoSwarmSim. We have delved into the challenges we faced when converting Python logic to Go, and we have highlighted how we overcame them. Furthermore, we have presented specific changes made to enhance the code's clarity, readability, and efficiency. These changes encompassed several features of PySwarmSim, such as forgiveness, waiting, retry, and the responsible node calculation algorithm. In addition to the improvements made to existing features, GoSwarmSim introduces new concepts, such as the epoch, which have been seamlessly integrated into the existing design and features. Through these modifications, we have successfully improved GoSwarmSim's performance while also better reflecting and simulating Swarm[24]. By demonstrating the benefits of these changes, we hope to offer useful insights for developers working on similar projects.

Chapter 6

Evaluation

In this chapter, we present a comprehensive evaluation of GoSwarmSim that we have developed, aiming to compare its performance with PySwarmSim. By evaluating, we aim to provide a better understanding of the capabilities and effectiveness of our simulation tool and its potential advantages over PySwarmSim.

6.1 Validation

In the process of rewriting and making improvements to any code, it is important to ensure that the new implementation provides the intended results. This is also the case for our project. We were meant to make the PySwarmSim faster and better, but the results should remain mostly similar. The research using PySwarmSim presents several results [16], which we also should be able to produce with GoSwarmSim.

To ensure that GoSwarmSim produces equivalent results to PySwarmSim, we developed analysis functions that replicated the figures presented in [16]. Although we anticipated slight differences in the results because of the inherent randomness of both simulation tools. However, we found the calculations derived from GoSwarmSim's output were indeed consistent with those obtained from PySwarmSim. Furthermore, the ratio of threshold failures, access failures, and successes was within the margin of error when simulating with the same parameters in both versions of the simulation. Therefore, it is reasonable to state that we initially succeeded in replicating PySwarmSim in a different programming language.

However, as detailed in Chapter 5, GoSwarmSim introduced several changes to the features and implementations of the simulation tool compared to PySwarmSim. For instance, epochs were introduced in Section 4.4, and waiting and retry were reimaged in Section 5.7. The purpose of these modifications was to make our simulation tool more closely reflect Swarm [24] and increase simulation accuracy. These changes naturally altered the end result and output in GoSwarmSim compared to PySwarmSim, which we see as a positive outcome. If there were no changes, our implementation improvements would not have mattered. Ultimately, our new results provide a more realistic simulation of Swarm, albeit with differences from PySwarmSim, which can make direct comparisons more challenging.

6.2 Performance Review

In the upcoming sections, we will make a direct comparison between the simulation tools based on performance in the form of execution time. First, PySwarmSim is compared to our simulation, and then different versions of GoSwarmSim are compared to each other to view how concurrency further increases performance.

When comparing the execution time of two programs, it is important to keep in mind that the execution time can vary substantially depending on the hardware and software environment in which the programs are run. Factors such as CPU clock speed, memory size, and operating system configuration can all affect the performance of a program. The performance review in the upcoming sections is based on the following hardware:

- Intel(R) Core(TM) i7-10700 CPU @ 4.0GHz sustained all core boost
- 16GB RAM

6.2.1 PySwarmSim vs GoSwarmSim

Python and Go are two very different programming languages. Python is known for its simplicity and readability [11], while Golang is designed for efficiency and concurrency [8]. This section will perform a comparison based on execution time

on the two different simulations. Firstly, we will compare a version of GoSwarmSim that produces output in a format similar to PySwarmSim, and then we will compare a version that is optimized for output.

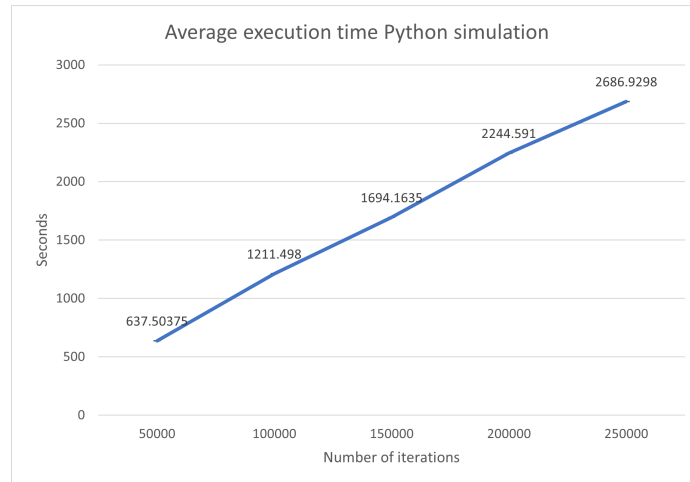


Figure 6.1: Average execution time of Python simulation, 10 runs

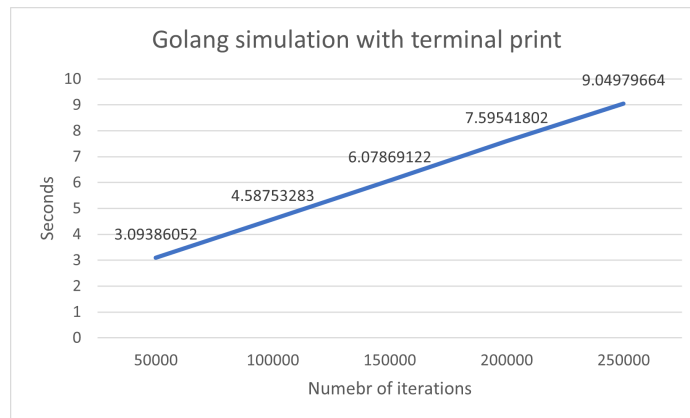


Figure 6.2: Average execution time of Go simulation with output same as Python version, 16 goroutines, 10 runs

In the PySwarmSim, the output from the simulation is printed directly to the terminal. In order to make them more comparable, similar adjustments were made to GoSwarmSim, shown in Figure 6.1 and Figure 6.2. In this comparison, as well as the others in this section, we have chosen to limit the number of iterations from 50,000 to 250,000 due to the significant increase in execution time

that would be required when running PySwarmSim with a higher number of iterations. To ensure the measurements correctly reflect reality, ten executions were performed for each result with the same number of iterations, and the average was taken as the final result. The result shows that there is a very big difference in execution time for the two programming languages, as during these tests the implementations of the two tools were basically identical.

As we can see from the two figures, the execution time of the Go version is more than 200 times faster than the Python version for 50,000 iterations, increasing to almost 300 times faster for 250,000 iterations. That is, the rate of increase in time for a higher number of iterations is lower for the Go version indicating that this implementation also has better performance scalability.

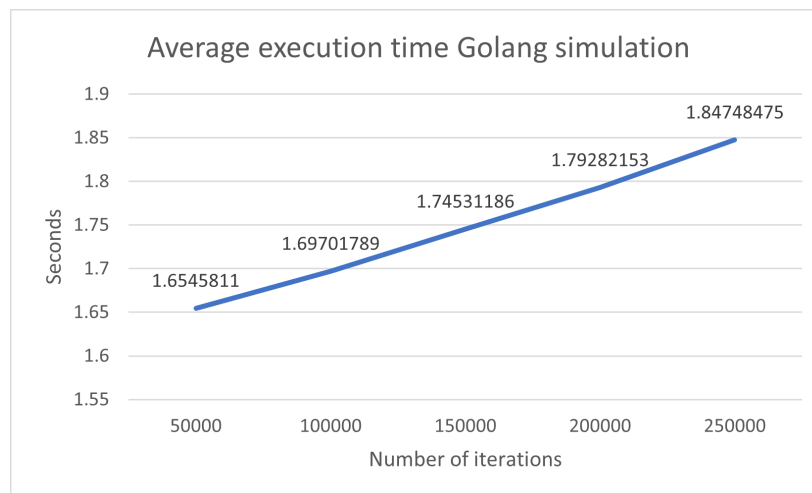


Figure 6.3: Average execution time of Go simulation with optimized output, 16 goroutines, 10 runs

To further speed up our simulation, we looked at different ways of saving the simulation output. We noticed that printing output directly to the terminal was a demanding task. To minimize the performance impact of continuously writing to the terminal, we decided to write the output to a file using Go's `bufio` package [6], as described in Section 4.6.1. This enabled efficient and buffered writing which reduced the overhead associated with writing directly to the terminal and resulted in even faster execution times, shown in Figure 6.3. This figure shows an even bigger performance improvement from Python to Go where at 50,000 iterations the Go implementation is approximately just less than 400 times faster,

increasing to an impressive 1450 times faster at 250,000 iterations.

Because of how slow the PySwarmSim is at 250k iterations, running it with for example 10 million iterations will take an incredibly long time. Using Figure 6.1, we can instead try to calculate what the execution time would have been for a higher number of iterations. Even though this might not be entirely correct, this calculation allows us to make an approximation of how long the execution time would be for a higher number of iterations, assuming the graph will continue in a linear fashion. The increase rate for the graph can be calculated by taking the difference between each consecutive pair of numbers and taking the average of the differences:

$$\frac{573.99425 + 482.6655 + 550.4275 + 442.3388}{4} = 512.3565125 \quad (6.1)$$

This indicates how many seconds the execution time grows for every 50,000 iterations. To find approximately the execution time for 10 million iterations we do this calculation:

$$\frac{512.3565125}{50,000} \times 10,000,000 = 102,471.3025 \quad (6.2)$$

The result suggests that for 10 million iterations, PySwarmSim would take over 100,000 seconds, which converts to approximately 28 hours, or 1.2 days. With the same configuration settings and the number of iterations, GoSwarmSim comparatively completes in just over 10 seconds, as shown in Figure 6.6.

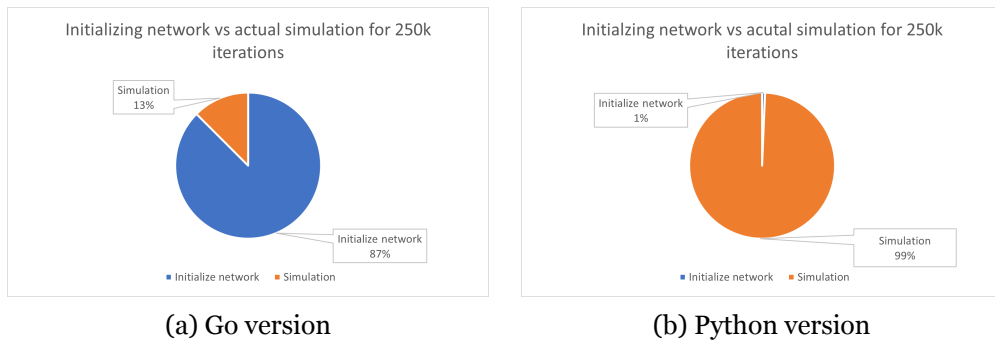


Figure 6.4: How much of the total execution time goes towards initializing the network for both versions

An important observation from Figure 6.4a is that when simulating with 250,000

iterations in GoSwarmSim, 87% of the total execution time is dedicated to network initialization. As the total execution time of GoSwarmSim was 1.85 seconds, we can conclude that the simulation tool uses a constant time of approximately 1.6 seconds for each simulation run with a network size of 10,000 nodes, regardless of the number of iterations. The initialization of the network relies on algorithms that are not as optimized as the main loop, which takes advantage of concurrency. As a result, network initialization becomes a constant time sink for simulation runs, regardless of the number of iterations simulated.

The relatively large initialization time is a result of the two processes involved in network initialization: creating the network nodes from the network file and creating the graph with the edges between the nodes. Both processes use algorithms with time complexity of $O(n^2)$, meaning that the increase in time spent is exponential to the size of the network and graph.

This impacts the comparison of execution times between the two versions, where in PySwarmSim, the network initialization is only 1% of the overall execution time in 6.4b. If we were to optimize the network initialization, the execution time of the Go version would become even faster and would have greatly impacted the comparison between the two languages for the chosen number of iterations. If we were to remove the time taken to initialize the network from both simulations, we observe a significant increase in performance boost. From Figure 6.4, removing 87% of the execution time from GoSwarmSim in Figure 6.3 and 1% from PySwarmSim in Figure 6.1 makes our simulation approximately 10,000 times faster. Which is the performance increase GoSwarmSim approaches when continuously increasing the number of iterations, minimizing the impact initialization has on the total execution time.

6.2.2 Different versions of GoSwarmSim

This section aims to compare various versions of GoSwarmSim and evaluate the impact of varying the number of goroutines and iterations on the simulation performance.

First, we will look at how the minimum amount of goroutines in our current version compares to using 16 workers and additionally when using no concurrency at all. As previously explained in Chapter 4, the minimum number of goroutines required for the final version of GoSwarmSim is two, with the request-

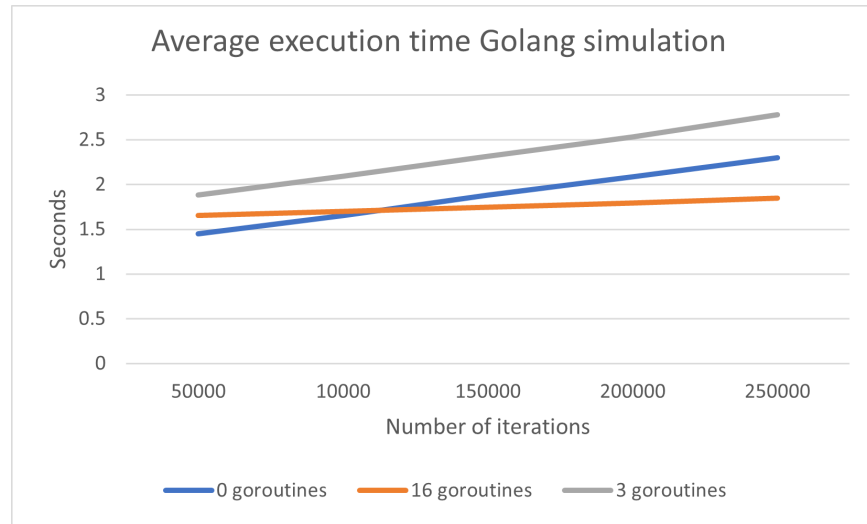


Figure 6.5: Comparison between no concurrency, concurrency with 3 goroutines, and with 16 goroutines

Worker and a single routingWorker. However, in order to generate useful output, the outputWorker needs to be enabled as well.

Given that our test machine has 16 available CPU threads, it seems like an obvious choice to set the number of total goroutines to 16, without further research on what the optimal number should be. This is also the current default setting for the number of goroutines, as the program dynamically adjusts the number of goroutines to match the maximum number of threads available on the machine it's running on. To test the impact of concurrency on the performance of GoSwarm-Sim, we used an earlier version of the program that was designed without any concurrency.

In Figure 6.5, a comparison is made between 0 goroutines (no concurrency), 3 goroutines, and 16 goroutines for iterations ranging from 50,000 to 250,000. The figure shows an interesting observation: for iterations below 100,000, the non-concurrent version is actually faster than the versions using 3 and 16 goroutines. This is partly due to all versions needing to initialize the same network which is done in a non-concurrent way and accounts for a significant portion of the total execution time. Along with the overhead introduced when initializing all the workers in the concurrent versions also increases the execution time compared to the non-concurrent version.

The same figure also indicates that for 3 goroutines, the execution time increases at a similar rate to the non-concurrent version for these iterations, thereby illustrating the overhead associated with concurrency. When using several goroutines, specifically 16 in this scenario, the execution time improves when the number of iterations increases. We also note that the 16-goroutine version shows a significantly lower rate of increase in execution time compared to the other versions. This suggests that for a larger amount of iterations, the differences in execution time will be more significant. While GoSwarmSim with 0 and 3 goroutines are already substantially faster compared to PySwarmSim, as shown in Figure 6.1, it is equally interesting to observe the additional performance increase achieved by adding more goroutines.

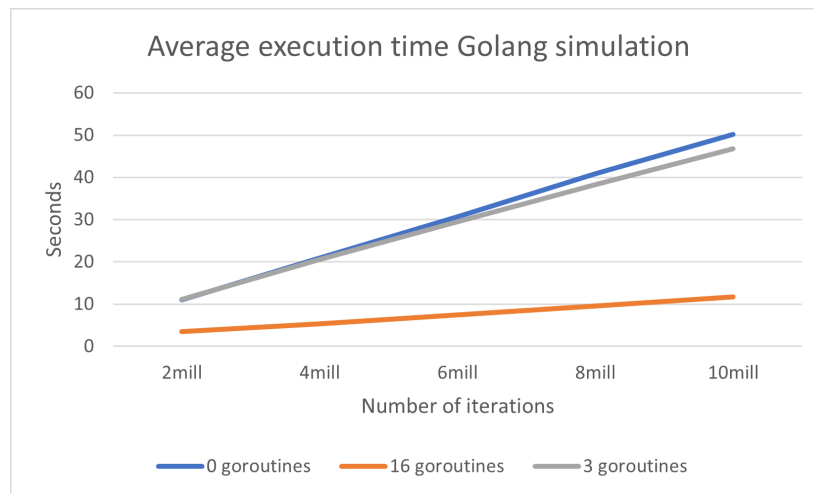


Figure 6.6: Comparison of no concurrency, concurrency with 3 goroutines and with 16 goroutines

Since GoSwarmSim has a much faster execution time, we can perform tests with a significantly higher number of iterations. Looking at Figure 6.6, when running 10 million iterations, the version with 16 Goroutines is around 3.5 times faster, compared to only 1.5 times faster in Figure 6.5, at 250,000 iterations. This shows that as the number of iterations increasingly grows, resulting in longer execution times, concurrency also plays an increasingly important role. It is also worth noting that the version with 3 goroutines becomes measurably faster than with no concurrency when running the simulation with this many iterations.

This observation highlights the importance of concurrency as the simulation

time increases. Additionally, it is interesting to note that the version with 3 goroutines performs significantly better than the non-concurrent version when running the simulation with a large number of iterations. This suggests that even a modest level of concurrency can lead to notable performance gains.

6.2.3 Finding the optimal number of goroutines

To determine what the optimal number of goroutines is, we have experimented with different numbers of goroutines between 6 and 100. Each number of goroutines has been tested with 50 runs each and with 10 million iterations in each simulation run. This is done because the difference in execution time is minimal for certain goroutines, and several runs help converge the execution time toward its actual value. The total number of goroutines used by the program includes the `requestWorker` and the `outputWorker` in addition to the `routingWorkers`. However, since the `routingWorkers` are the only ones that can scale, it's important to note that if the simulation is run with a total of 10 goroutines, only 8 of them would be `routingWorkers`.

Additionally, as explained in 4.6.1, `GoSwarmSim` implemented the possibility of doing calculations on the data in memory. Without this approach, the results would have to be calculated after the simulation, which would increase the total time required to retrieve useful results. For most of the possible calculations, the reduced performance is not noticeable. However, one of the calculations which compute the reward fairness in the program becomes a bottleneck when activated. Consequently, that calculation was disabled for the following tests.

Based on the results presented in Figure 6.7, it can be observed that the execution time shows a decreasing trend from 6 goroutines to 13 goroutines, after which it starts increasing again. This trend suggests that using 13 goroutines provides the fastest simulation performance, making it the optimal number of goroutines with our hardware. Although, it's worth noting that the difference between 13 and 26 goroutines is minimal with less than a 10% increase in execution time.

To test the performance of `GoSwarmSim` with even higher numbers of goroutines, we ran simulations with 50 and 100 goroutines. The graph shows that the difference between 26 goroutines and 50 goroutines is minimal, but still an overall increase in execution time. Moreover, the execution time for 100 goroutines sees a larger jump from 50 to 100, compared to 26 to 50.

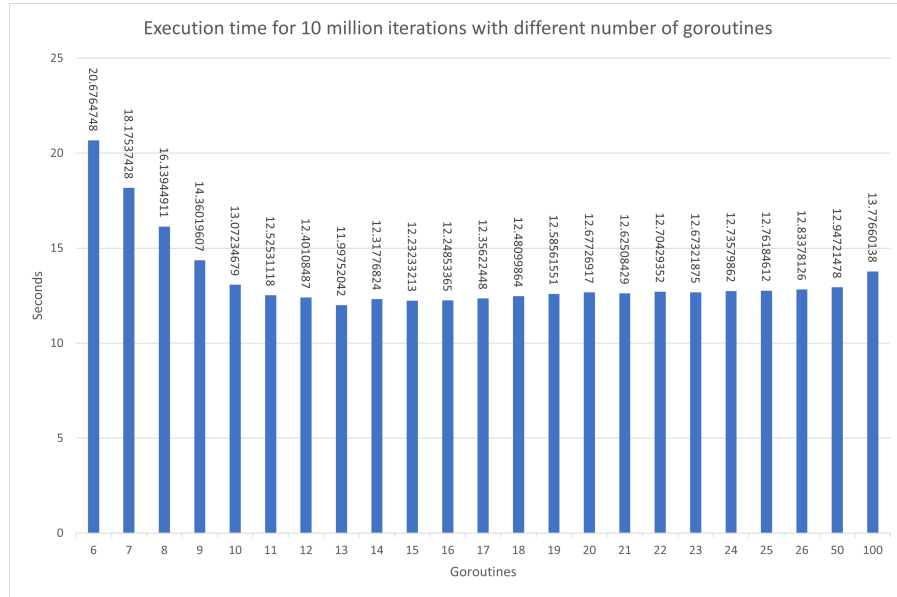


Figure 6.7: Average execution time for 10 million iterations with different numbers of goroutines

Figure 6.7 illustrates that increasing the number of goroutines does not always result in faster execution times, and determining the optimal number of goroutines can be challenging. Additionally, the graph shows that the optimal number of goroutines for this simulation is not exactly the number of available CPU cores, but rather a slightly lower number. However, this does not necessarily mean that is always the best approach for all users of GoSwarmSim. The optimal number of goroutines will depend on various factors such as the hardware used.

Nonetheless, when comparing the differences between 8 and 13 goroutines to 13 and 26, we see a larger difference in execution time in the former. This means we can generally conclude in a situation with many variables, it's better to have too many goroutines than too few. As a result, based on our findings, we have determined that setting the number of goroutines equal to the number of threads available on the machine is generally recommended to minimize the chance of enabling too few goroutines. Therefore, we have made this the default setting in our configuration. While in our specific use case, we found that 13 goroutines are optimal, users can adjust this setting according to their machine's capabilities.

6.2.4 Larger Networks

The other goal of this project, besides making the simulation tool faster, was to make a better simulation tool that could handle bigger networks. GoSwarmSim makes this possible without sacrificing too much performance.

One of the challenges with running the simulation on bigger networks is creating the network file. The algorithm used for creating the network file has not been optimized and can potentially be improved. Even with GoSwarmSim, the algorithm still has limitations in terms of the size of the network that can be created due to execution time and memory constraints on the system. Looking at Figure 6.8, we can see how the network file creation scales with different numbers of nodes. Because of how the network is created, larger networks are exponentially slower to make. We still observe that using Go makes network creation considerably faster. In its current state, GoSwarmSim is still limited by the size of the network that can be created due to the exponential growth in time.

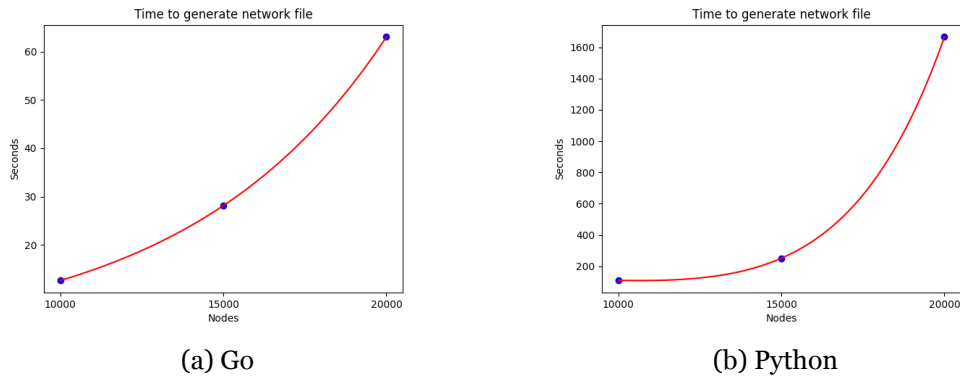


Figure 6.8: Time taken to make network file for 10k, 15k and 20k nodes

In all of the comparisons during this chapter, we have used the default values of a 16-bit address space and a network of 10,000 nodes inherited from PySwarmSim. The network used in the comparison has an address space of 20 bits and a size of 40,000 nodes. Running PySwarmSim and GoSwarmSim on the new larger network made the execution time noticeably slower. When running the simulations at 250,000 iterations we observed an increase in execution time of approximately 1000% in PySwarmSim and an increase of around 500% in GoSwarmSim.

As such, we observed the initialization process became a bigger percentage

of the total execution time in GoSwarmSim than it was in Figure 6.4a, and similarly increasing the total number of iterations would reduce the percentage values. This implies that it is not the simulation process itself that is affected by the larger network, but mostly the initialization of the network.

This was not the case for PySwarmSim. The time needed for the initialization process also increased, but not to the same extent in regards to the total execution time as GoSwarmSim. Implying that not only was the initialization layer impacted, but also the other layers in the simulation. These observations make it fair to state that our GoSwarmSim handles larger networks better than PySwarmSim.

For the remainder of this section, we will provide some comparisons of the findings from the original research in [16], with results from GoSwarmSim with the larger network. The evaluation presented here does not delve into a detailed discussion of why some results may vary on larger networks, but it aims to highlight the importance and potential value of having a faster simulation and the ability to handle larger networks in further research.

Figure 6.9 presents the average number of hops required to traverse each route in a simulated run as the bin value k varies. The top graph is retrieved from the Python research conducted in [16], and the bottom is created using the GoSwarmSim with the larger network and a bigger address space. Specifically, PySwarmSim is using the 16-bit 10k network and is compared to GoSwarmSim's 20-bit 40k network. The graph illustrates that the average number of hops grows

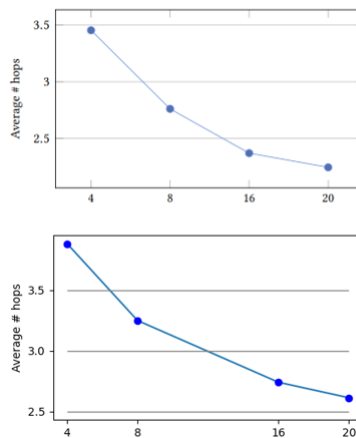


Figure 6.9: Comparison of the average number of hops for every route

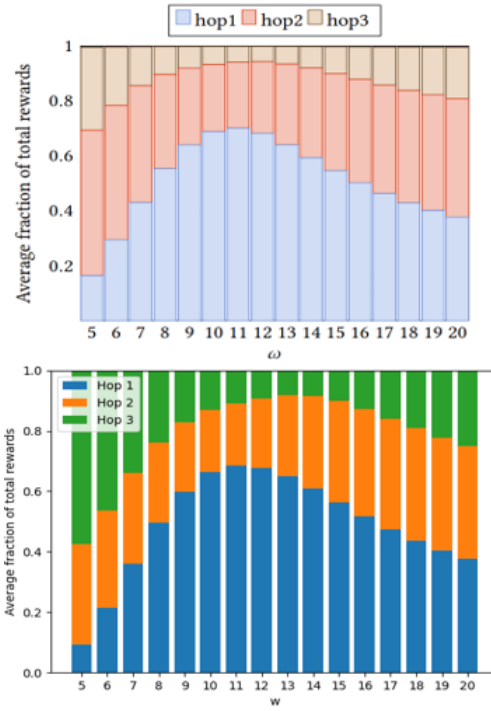


Figure 6.10: Comparison of the average fraction of rewards received on different hops in a route for different values of the max proximity order

with the network size and larger address space. This may be because of the possibility of larger distances between the requester nodes and the desired chunks, which results in more hops being taken to reach the desired destination.

A comparison of the average fraction of rewards that are received on the different hops in a route is presented in Figure 6.10. The average fraction of rewards is calculated for values of a variable called omega. Omega is the maximum cost for a node to request a chunk, and rewards are parametrized by this variable. Different values for omega are used to experiment with how rewards are distributed in a route [16]. This figure also presents the result of PySwarmSim at the top and the results from GoSwarmSim at the bottom. The Figure shows the largest differences for lower values of omega when increasing the network size, but there is still some discrepancy throughout the whole graph. One observation is that a larger network with a larger address space rewards the last hop in a route more for every value of omega.

6.3 Conclusion on Evaluation

The question of whether the time spent designing and implementing concurrency was worth it is a valid one. Figure 6.5 shows that, even without concurrency, GoSwarmSim performs similarly to the version with concurrency for small numbers of iterations. However, as we increase the total iterations and grow the networks past what was possible with PySwarmSim, the addition of concurrency makes sense and provides substantial improvements. Ultimately, considering that GoSwarmSim with concurrency is slightly faster for small numbers of iterations and scales considerably better when the total number of iterations is increased, we can comfortably conclude that the design and implementation of concurrency was a good choice and effectively made the simulation tool faster.

Depending on the number of iterations and the number of goroutines used, GoSwarmSim demonstrated speed gains ranging from 1,450 to 10,000 times faster than PySwarmSim. This means that it is not only the usage of Go and the use of concurrency that makes this tool faster but also because of specific implementation improvements. It is also finally worth noting, that while implementing the simulation to be faster, it also has become more accurate and closer to how Swarm [25] actually is implemented.

Chapter 7

Discussion

The aim of this discussion is to explore the impact of the redesign on GoSwarmSim and consider how it would have differed if not built upon PySwarmSim. Additionally, we shall address the limitations of our work and suggest directions for future work.

7.1 GoSwarmSim From Scratch

In the beginning stages of developing GoSwarmSim, we merely copied the design of PySwarmSim. Replicating the design provided a foundation for improving both the design and implementation. How would the design of GoSwarmSim differ if it had not been based on the design of PySwarmSim?

Firstly, GoSwarmSim would aim to closely replicate the functionality of Swarm. However, since GoSwarmSim is a simulation of bandwidth incentives, it may not necessarily need to completely replicate all aspects of Swarm’s functionality but rather simulate them. For example, one aspect is the variation in the number of nodes within the Swarm network. Due to the constant number of nodes in the network throughout the simulation, GoSwarmSim is not affected by free-riding and network churn something that is a challenge in Swarm [18].

Secondly, we think that developing GoSwarmSim from scratch would likely result in a design approach similar to the one we adopted. Nonetheless, without needing to replicate cadCAD’s functionality, the simulation steps in GoSwarmSim would most likely have been different. One design could be entirely different by having each individual node be its own goroutine creating, handling the re-

questing of chunks, and forwarding requests for a chunk of their own accord by communicating with neighboring nodes directly.

This would allow the nodes to work similarly to real-world peers, but instead of talking over the internet, they would communicate through Go channels. Additionally, using channels would eliminate the need for global states and locks since state information would be stored on each individual node, and channels' inherent thread-safe nature would keep the states consistent [8]. While this could be a more realistic simulation of Swarm [24], it might also have a large downside in the potentially large overhead by having a goroutine for each node in a network consisting of thousands of nodes. However, this might again be avoided by having multiple goroutines sharing the load of multiple nodes.

All in all, exploring different approaches, including the ones mentioned, would have been an interesting avenue to pursue.

7.2 Future Work

Considering the limitations of time and resources, this project has successfully met its objectives while achieving excellent results. There are, however, always things that may use more work and development, just like with any project. In this section, we present a few possibilities for future work that, given additional time, might improve the project's completeness and effectiveness.

Tests and improving error handling

Initially, we placed a significant emphasis on creating tests as they were helpful in verifying whether the results generated by GoSwarmSim were consistent with the expected results from PySwarmSim. However, over time, the creation and maintenance of tests took a back seat in favor of other improvements to the simulation tool. As we made changes to the implementation, the resulting output began to differ from that of PySwarmSim, which required constant adjustment of the tests. In the future, it would be wise to reinstate the tests and incorporate more of them.

Similarly, the addition of enhanced error handling would be beneficial for future work. Although we currently catch numerous errors, there are still instances where the simulation breaks after making certain changes, requiring us to de-

bug the code to pinpoint the issue. Therefore, prioritizing the addition of more catches for edge cases and appropriately handling errors would be wise.

Improving incorporation of config.yaml

The addition of the config.yaml file to contain the simulation configurations was a positive move. Keeping it separate from the codebase, conforming to the standards of other programs, and making it simple to read and edit. However, the integration with the simulation code could have been improved. Moving back to the error handling point, if the name of some fields in the yaml file is changed, that causes the simulation to reset that setting to default. making it challenging to identify the error.

Additionally, the main idea behind the config.yaml was to easily switch between configuration settings and experiments. Nevertheless, we failed to include the feature of having multiple yaml files. Having the ability to use a config file as an argument to the simulation would be beneficial, allowing the user to have multiple yaml files with different settings and quickly switch between them, instead of manually editing one or more fields from the single config.yaml file. Therefore, the feature of having multiple yaml files would be a valuable addition to the current implementation.

Optimize create network file

The larger network that has been used in Section 6 is still small compared to what a real network can potentially look like. Although we are able to make a bigger network, GoSwarmSim is also limited in how big of a network it can make. This is because the method for creating the network uses a very complex algorithm which makes the creation of a network extremely slow. We did not recognize this bottleneck in the early stages, so the network creation is essentially a direct translation of the Python network creation. Because of the complex algorithm, running out of memory when creating the network was also an issue we encountered if address spaces were too big.

We explored some ways to utilize concurrency in the later stages of the project to create the network but encountered some discrepancies in the network file when doing this compared to the original way. Because of these limitations, exploring different methods for creating the network will be something for the fu-

ture of this simulation tool, where recreating the whole network creation algorithm probably would be the best option.

Optimize initialize network

Initializing the network from the network file relies on a similar complex algorithm as creating the network. This was also something that we in the early stages did not recognize as important, and is done in the same way as it is in PySwarmSim. Because of this, initializing the network becomes a problem in GoSwarmSim. As seen in Figure 6.4, the network initialization is a far bigger portion of the total execution time in GoSwarmSim, which makes it a bigger bottleneck for our simulation tool. The severity of the time taken initializing the network was something that was not noticed until the later stages of our project.

Output

The output method presented in Section 4.6.1, still has some limitations when doing calculations in memory. First of all, it does not handle all of the calculations presented in [16]. This is because we did not have time to produce every experiment as this was not the priority in our work, just more of an addition to make the simulation more user-friendly.

Secondly, some of the experiments can be quite CPU-intensive. If they are, calculating results in memory can essentially stop the whole simulation while they are being calculated. This is because the output needs to be written to a file before the simulation can move on. We have an idea of how this can be fixed by utilizing concurrency, but as stated earlier, this was not the priority and has therefore not been done yet. Although calculating some of the experiments in memory is slow, there is still the possibility to skip the calculation and just print the output. Then, the desired calculations can be done after the simulation is done.

Chapter 8

Conclusion

This paper introduces GoSwarSim, a simulation tool developed to enhance the simulation of bandwidth incentives in Swarm by rewriting an existing tool originally written in Python. By utilizing Go, the tool's design and implementation were enhanced, leading to several improvements in its performance. One significant improvement was the use of Go's concurrency and the adoption of a workers-based approach, resulting in faster execution times. The implementation was further optimized by specific enhancements meant to improve the simulation's performance and make it more reflective of Swarm. Overall, GoSwarSim offers faster simulation speeds, improved accuracy, and user-friendliness, making it a valuable tool for researchers and developers in the field.

Appendix A

Instructions to Compile and Run System

Github link: <https://github.com/Swarm-Bachelor/go-incentive-simulation>

Ensure Golang, preferably version 1.19.5 or later, is installed on the computer.

Clone the GitHub repository.

```
$ git clone git@github.com:Swarm-Bachelor/go-incentive-simulation.git
```

Change directory to ./go-incentive-simulation.

```
$ cd go-incentive-simulation
```

Configure the settings for the simulation by editing the config.yaml file.

```
$ nano config.yaml
```

Run the program.

```
$ go run main.go
```

View output in terminal and in ./results folder

```
$ cd results
```

```
$ cat *fileName*.*extension*
```

Generate new network files, using config.yaml for settings

```
$ cd data
```

```
$ go run generate_data.go
```


Bibliography

- [1] oxPhillan. *Swarm: an Ethereum-based P2P Storage Protocol*. 2022. URL: <https://web3edge.io/research/what-is-swarm/>.
- [2] AltexSoft. *The Good and the Bad of Python Programming Language*. URL: <https://www.altexsoft.com/blog/python-pros-and-cons/>. accessed: 09.05.2023.
- [3] Remzi H. Arpaci-Dusseau and Andrea C. Arpaci-Dusseau. *Operating Systems: Three Easy Pieces*. 1.00. Arpaci-Dusseau Books, Aug. 2018.
- [4] Binance.com. *What Is Decentralized Storage?* 2023. URL: <https://academy.binance.com/en/articles/what-is-decentralized-storage>. accessed: 03.23.2023.
- [5] Vincent Blanchon. *Go: How to Reduce Lock Contention with the Atomic Package*. 2020. URL: <https://medium.com/a-journey-with-go/go-how-to-reduce-lock-contention-with-the-atomic-package-ba3b2664b549>.
- [6] *Bufio Package Documentation*. URL: <https://pkg.go.dev/bufio>. accessed: 09.04.2023.
- [7] cadcad. *cadCad Documentation*. URL: <https://github.com/cadCAD-org/cadCAD/tree/master/documentation>.
- [8] Alan A. A. Donovan and Brian W. Kernighan. *The Go Programming Language*. Pearson Education, 2015. ISBN: 0134190564.
- [9] Filecoin. *Cloud Storage Services: Understanding the Decentralized Storage Approach*. 2023. URL: <https://filecoin.io/blog/posts/cloud-storage-services-understanding-the-decentralized-storage-approach/>. accessed: 09.05.2023.

- [10] Gabbi Fisher and Christopher Yeh. “Comparing Producer-Consumer Implementations in Go, Rust, and C”. In: (2017), p. 7. URL: <https://stanford-cs242.github.io/f17/assets/projects/2017/gsfisher-chrisyeh.pdf>.
- [11] Tony Gaddis. *Starting Out With Python*. Pearson Education, 2018. ISBN: 9781292225753.
- [12] github.com. *cadCAD simulation engine*. URL: <https://github.com/cadCAD-org/cadCAD>. accessed: 26.04.2023.
- [13] Go. *Documentation*. URL: <https://go.dev/doc/>. accessed: 09.05.2023.
- [14] Rinke Hendriksen. *Understanding Swarm’s bandwidth incentives*. 2021. URL: <https://medium.com/ethereum-swarm/understanding-swarms-bandwidth-incentives-169f4d439675>.
- [15] Lumitim Jason B. Alonso and Robyn Speer. *ordered-set 4.1.0*. 2022. URL: <https://pypi.org/project/ordered-set/>.
- [16] Leander Jehl and Vahid Heidaripour Lakhani. “Tit-for-Token: Fair Rewards for Moving Data in Decentralized Storage Networks”. In: (2023), p. 21.
- [17] Jamal Kaksouri. *A Comprehensive Guide to Pointers in Go*. 2023. URL: <https://medium.com/@jamal.kaksouri/a-comprehensive-guide-to-pointers-in-go-4acc58eb1f4d>.
- [18] Vahid Heidaripour Lakhani et al. “Fair Incentivization of Bandwidth Sharing in Decentralized Storage Networks”. In: *2022 IEEE 42nd International Conference on Distributed Computing Systems Workshops (ICDCSW)*. IEEE. 2022, pp. 39–44.
- [19] Petar Maymounkov and David Mazieres. “Kademlia: A peer-to-peer information system based on the xor metric”. In: *Peer-to-Peer Systems: First International Workshop, IPTPS 2002 Cambridge, MA, USA, March 7–8, 2002 Revised Papers*. Springer, 2002, pp. 53–65.
- [20] Serhii Osadchuk. *Go vs Python in 2023: Which One to Choose?* 2023. URL: <https://doit.software/blog/go-vs-python>. accessed: 01.05.2023.
- [21] Python. *heapq — Heap queue algorithm*. n.d. URL: <https://docs.python.org/3/library/heapq.html>. accessed: 07.02.2023.

- [22] Thierry S. *Why we switched from Python to Go*. 2017. URL: <https://getstream.io/blog/switched-python-go/>. accessed: 01.05.2023.
- [23] Daniel Dominic Sleator and Robert Endre Tarjan. “Self-Adjusting Binary Search Trees”. In: *J. ACM* 32.3 (July 1985), pp. 652–686. ISSN: 0004-5411. DOI: 10.1145/3828.3835. URL: <https://doi.org/10.1145/3828.3835>.
- [24] Viktor Trón. “The Book of Swarm: storage and communication infrastructure for self-sovereign digital society back-end stack for the decentralised web”. In: *V1. o pre-Release 7* (2020).
- [25] Viktor Trón et al. “Swap, swear and swindle: incentive system for swarm”. In: *Ethereum Orange Paper* (2016).
- [26] Guido VanRossum and Fred L Drake. *The python language reference*. Python Software Foundation Amsterdam, Netherlands, 2010.
- [27] Liangzhou Wang and Chaobin Wang. “Producer-consumer Model Based Thread Pool Design”. In: *Journal of Physics: Conference Series* 1616.1 (Aug. 2020), p. 012073. DOI: 10.1088/1742-6596/1616/1/012073. URL: <https://dx.doi.org/10.1088/1742-6596/1616/1/012073>.
- [28] Khor Win Win. *The State of Decentralized Storage*. URL: <https://www.coingecko.com/research/publications/the-state-of-decentralized-storage>. accessed: 09.05.2023.

4036 Stavanger
Tel: +47 51 83 10 00
E-mail: post@uis.no
www.uis.no

© 2023 Filip B. Gotten, Rasmus Øglænd, and Torjus J. Knudsen